



BRNO UNIVERSITY OF TECHNOLOGY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF INFORMATION SYSTEMS

ÚSTAV INFORMAČNÍCH SYSTÉMŮ

RAG APPLICATION OVER A MANAGED KNOWLEDGE BASE

RAG APLIKÁCIA NAD SPRAVOVANOU ZNALOSTNOU BÁZOU

BACHELOR'S THESIS

BAKALÁŘSKÁ PRÁCE

AUTHOR

AUTOR PRÁCE

PATRIK KIŠEDA

SUPERVISOR

VEDOUCÍ PRÁCE

Ing. VLADIMÍR BARTÍK, Ph.D.

BRNO 2026

Zadání bakalářské práce



173019

Ústav: Ústav informačních systémů (UIFS)
Student: **Kišeda Patrik**
Program: Informační technologie
Název: **RAG aplikace nad spravovanou znalostní bází**
Kategorie: Umělá inteligence
Akademický rok: 2025/26

Zadání:

1. Prostudujte problematiku velkých jazykových modelů (LLM), techniky pro ovlivnění jejich chování a výstupů, retrieval-augmented generation (RAG) a agentních systémů.
2. Analyzujte požadavky na RAG aplikaci pracující s existující znalostní bází. Zvolte vhodné technologie pro implementaci.
3. Navrhněte architekturu RAG aplikace, proces zpracování dat a obsluhu požadavků a perzistentní uložení dat.
4. Po konzultaci s vedoucím implementujte navrženou aplikaci.
5. Otestujte funkčnost implementované aplikace.
6. Zhodnotte dosažené výsledky.

Literatura:

- Alto, V.: Building LLM Powered Applications: Create Intelligent Apps and Agents with Large Language Models. Packt Publishing, 2024. ISBN: 978-1835462317.

Při obhajobě semestrální části projektu je požadováno:
Body 1-3.

Podrobné závazné pokyny pro vypracování práce viz <https://www.fit.vut.cz/study/theses/>

Vedoucí práce: **Bartík Vladimír, Ing., Ph.D.**
Vedoucí ústavu: Kolář Dušan, doc. Dr. Ing.
Datum zadání: 1.11.2025
Termín pro odevzdání: 13.5.2026
Datum schválení: 24.10.2025

Abstract

This thesis designs and implements a Retrieval-Augmented Generation application for querying a managed knowledge base. The system ingests source documents, extracts and chunks their content, stores metadata and lexical search data in SQLite, stores dense vectors in Qdrant, and supports dense, lexical, and hybrid retrieval. Retrieved chunks are assembled into grounded prompts for a large language model through the LiteLLM library and generated answers are returned with source identifiers. The prototype is verified by automated tests and controlled experiments comparing retrieval modes, chunk sizes and answer traceability evaluation. The result is an end-to-end system whose responses are enhanced with source traceability and improved answering capacity.

Abstrakt

Táto práca navrhuje a implementuje Retrieval-Augmented Generation(RAG, generovanie rozšírené o vyhľadavanie) aplikáciu, ktorá zlepšuje kvalitu a dôveryhodnosť odpovede veľkých jazykových modelov pomocou externej znalostnej bázy. Systém prijíma zdrojové dokumenty, extrahuje text a rozdeľuje ho do kusov, ukladá metadáta a textové vyhľadavanie do SQLite databázy, ukladá vektory zachytávajúce význam kusov vstupného textu do Qdrant databázy a podporuje lexikálne, sémantické a hybridné vyhľadavanie. Nájsené kusy dokumentov sa vkladajú do dotazu pre jazykový model cez LiteLLM knižnicu a následná odpoveď je vrátená a zobrazená spolu s identifikátormi zdrojov. Aplikácia je overená automatizovanými testami a riadenými experimentmi porovnávajúcimi režimy vyhľadávania, veľkosti kusov dokumentov a kvalitu odpovede s priloženým kontextom. Výsledky ukazujú funkčný systém, ktorého odpovede sú podložené zdrojmi obsiahnutými znalostnej báze.

Keywords

RAG, retrieval-augmented generation, knowledge base, vector database, LLM, language models

Kľúčové slová

RAG, retrieval-augmented generation, znalostná báza, vektorová databáza, LLM, jazykové modely

Reference

KIŠEDA, Patrik. *RAG application over a managed knowledge base*. Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. Vladimír Bartík, Ph.D.

Rozšířený abstrakt

Táto bakalárska práca sa zaoberá návrhom, implementáciou a overením Retrieval-Augmented Generation aplikácie nad spravovanou znalostnou bázou. Motiváciou pre prácu je snaha o zlepšenie odpovedacej schopnosti veľkých jazykových modelov pri úlohách, ktoré vyžadujú presné, aktuálne, alebo neverejné informácie. Samotný jazykový model odpovedá na základe znalostí získaných počas tréningu a na základe kontextu v dotaze. Ak potrebné informácie v tomto kontexte chýbajú, model môže vytvoriť neaktuálnu, neúplnú alebo nepodloženú odpoveď. Tento problém je viditeľný najmä pri interných dokumentáciách, technických špecifikáciách a znalostiach, ktoré sa často menia alebo nie sú verejne dostupné.

Retrieval-Augmented Generation rieši tento problém tým, že pred generovaním odpovede vyhľadá relevantné časti dokumentov v externej znalostnej báze a priloží ich k otázke používateľa ako kontext. Model potom nemusí spoliehať iba na svoje parametre, ale môže odpovedať podľa nájdených zdrojov. Dôležitou vlastnosťou takéhoto prístupu je aj možnosť priamej kontroly správnosti odpovede pomocou kusov textu nájdených v zdrojovej dokumentácii. Takýto systém vráti nielen samotnú odpoveď, ale aj zdrojové úryvky, používateľ vie overiť, z ktorých dokumentov bola odpoveď odvodená.

Cieľom práce bolo vytvoriť lokálny prototyp takejto aplikácie bez použitia RAG frameworku. Vlastná implementácia bola zvolená preto, aby boli jednotlivé časti spracovania dokumentov, indexovania, vyhľadávania, zostavenia promptu a generovania odpovede dostupné a testovateľné. Práca preto najprv opisuje relevantné technológie: veľké jazykové modely, ich obmedzenia, embeddingy, delenie textu na menšie časti, vektorové databázy, lexikálne vyhľadávanie, hybridné vyhľadávanie a nástroje, ktoré sa pri vývoji RAG systémov bežne používajú.

Navrhnutá aplikácia je organizovaná do vrstiev. Rozhranie aplikácie je postavené na FastAPI a jednoduchom webovom používateľskom rozhraní. Aplikačná vrstva zabezpečuje spracovanie dokumentov, extrakciu textu, kúskovanie textu, vytvorenie embeddingov, ukladanie dát, vyhľadávanie, zostavenie promptu a generovanie odpovede. Databázová vrstva kombinuje SQLite a Qdrant. SQLite uchováva metadáta dokumentov, obsah jednotlivých častí dokumentov, stav indexovania a lexikálny index pomocou FTS5. Qdrant uchováva sémantické vektorové reprezentácie kusov dokumentov a používa sa na sémantické vyhľadávanie.

Implementácia podporuje nahrávanie dokumentov vo formátoch PDF, TXT a Markdown. Po nahratí dokumentu sa vytvorí záznam o dokumente a indexovací proces. Samotné spracovanie prebieha na pozadí, aby používateľ nemusel čakať na dokončenie extrakcie, delenia a indexovania. Text sa po extrakcii normalizuje a následne rekurzívne delí. Tá sa snaží deliť text hierarchicky, napríklad odsekmi a riadkami a až potom podľa viet, medzier, alebo pevnej dĺžky znakov. Medzi susednými časťami sa ponecháva prekryv, aby sa znížilo riziko straty kontextu na hraniciach.

Vyhľadávacia časť aplikácie podporuje tri režimy. Sémantické vyhľadávanie používa embedding otázky samotnej a porovnáva ho s vektormi uloženými v Qdrante. Lexikálne vyhľadávanie používa SQLite FTS5 a hodnotenie BM25, takže je vhodné najmä pre presné výskyty slov a termínov. Hybridné vyhľadávanie kombinuje oba predchádzajúce režimy a ich výsledky spája pomocou Reciprocal Rank Fusion. Tento prístup umožňuje kombinovať sémantickú podobnosť s presným lexikálnym vyhľadávaním a poskytuje robustnejšie výsledky.

Po vyhľadaní relevantných častí dokumentov aplikácia zostaví prompt pre jazykový model. V režime so zahrnutým kontextom, prompt obsahuje pôvodnú otázku, nájdené úryvky, ich identifikátory a inštrukciu odpovedať iba podľa poskytnutého kontextu. Odpoveď

má používať citácie v tvare identifikátorov zdrojov, napríklad [S1] alebo [S2]. API následne vracia vygenerovanú odpoveď aj zoznam použitých zdrojov. Súčasťou implementácie je aj režim na kontrolu finálneho promptu, ktorý umožňuje zobrazit zostavený prompt bez volania jazykového modelu.

Testovanie systému prebehlo dvoma spôsobmi. Automatizované testy pokrývajú konfiguráciu, extrakciu textu, delenie dokumentov, ukladanie do SQLite a Qdrantu, lexikálne, sémantické a hybridné vyhľadávanie, zostavenie promptu, generovanie odpovedí a správanie API. Druhú časť overenia tvoria riadené experimenty na malej znalostnej báze. Tie porovnávali režimy vyhľadávania, sledovali vplyv veľkosti častí dokumentov a kontrolovali, či odpovede s vloženým kontextom obsahujú zdrojové identifikátory zodpovedajúce vráteným zdrojovým kusom.

Výsledkom práce je funkčný lokálny prototyp, ktorý dokáže prijať dokumenty, zaindexovať ich, vyhľadávať v nich viacerými spôsobmi a generovať odpovede podložené nájdenými zdrojmi. Experimenty neboli prevedené pomocou rozsiahleho benchmarku a nie je možné z nich vyvodzovať všeobecné tvrdenia o kvalite vyhľadávania na ľubovoľných dátach. Ukazujú však, že navrhnutá architektúra funguje od pôvodnej otázky až po odpoveď a že správanie systému je možné testovať a kontrolovať. Medzi hlavné obmedzenia patrí lokálne jednopoužívateľské zameranie, chýbajúca možnosť mazania a preindexovania dokumentov, vylepšené spracovanie úloh na pozadí a plynulý výpis odpovedí. Medzi možné rozšírenia patria: lepšia správa dokumentov, hlbšie testovanie na rozsiahlejších dátach, podporu dlhších konverzácií a využitie vyhľadávacej časti ako nástroja pre LLM agenta alebo vývojárskeho asistenta.

RAG application over a managed knowledge base

Declaration

I hereby declare that I have prepared this Bachelor's thesis independently under the supervision of Ing. Vladimír Bartík, Ph.D. I have listed all literary sources, publications, and other resources that I used in the preparation of this thesis. For language and stylistic refinement of the text and for consultation on the structure of certain parts of the thesis, I used the tools OpenAI Codex and Claude by Anthropic. I developed the program code independently; I used OpenAI Codex as a supporting tool for debugging, generating auxiliary code, refactoring, improving code comments, and consulting possible implementation approaches.

.....
Patrik Kišeda
May 12, 2026

Contents

1	Introduction	3
2	Overview of the current state of technologies	5
2.1	Large Language Models	5
2.2	Retrieval-Augmented Generation (RAG)	8
2.3	Tools and Abstraction Layers	13
3	Design and Technology Selection	15
3.1	Critical Evaluation of Existing Approaches	15
3.2	Requirements for the Proposed Application	16
3.3	System Architecture Design	17
3.4	Technology Selection and Justification	19
3.5	API Design	21
3.6	Testing and Evaluation Strategy	22
4	Implementation	23
4.1	Project Structure and Configuration	23
4.2	Document Ingestion Pipeline	24
4.3	Retrieval	28
4.4	Answer Generation	29
4.5	API Layer and Internal Application State	31
4.6	Frontend	32
4.7	End-to-End Example	32
4.8	Deviations from the Design	34
4.9	Known Limitations and Future Work	35
5	Testing	36
5.1	Testing Goals and Environment	36
5.2	Automated Tests	36
5.3	Controlled Retrieval Evaluation	37
5.4	Chunk Size Experiment	40
5.5	Grounded Answer and Source Traceability Check	41
5.6	Limitations of the Evaluation	43
6	Conclusion	44
	Bibliography	46

List of Figures

2.1	RAG pipeline architecture.	9
3.1	High-level system architecture. The FastAPI application communicates with Qdrant (running in Docker) for vector storage, SQLite for metadata and lexical search, and external LLM/embedding APIs through LiteLLM.	19
4.1	Recursive character chunking with separator cascade. Pass 1 splits the input recursively using the separator hierarchy until every piece fits within <code>base_size</code> . Pass 2 adds overlap by prefixing each chunk with the last <code>overlap</code> characters of the previous output chunk.	26
4.2	Most relevant SQLite values used by the application. The <code>documents</code> table stores source metadata and ingestion state; <code>chunks</code> stores retrievable text units and their embedding status; <code>chunks_fts</code> mirrors chunk text for BM25 lexical search; and <code>jobs</code> stores background indexing state, errors and summary statistics.	27
4.3	The grounded-mode prompt template. The first few lines constrain the model to the attached context, enforce citations and require the model to acknowledge when it had insufficient context. Each retrieved chunk is rendered as a block with its source identifier and metadata.	31
4.4	Web frontend of the implemented RAG application. Left: query view for submitting questions, selecting the retrieval mode, and inspecting generated answers with cited sources. Right: documents view for uploading source documents and checking their indexing status.	32

Chapter 1

Introduction

Large Language Models (LLMs) have in recent years demonstrated remarkable advances in natural language understanding and generation, achieving state-of-the-art performance in various tasks. For example, GPT-3 showed that scaling model size leads to improved performance and enables few-shot learning – the ability to perform new tasks with minimal examples. However, these models inherently face fundamental limitations when dealing with applications requiring precise and up-to-date information. They are constrained by their training data cutoff and may provide inaccurate, outdated, or even hallucinated information. This occurs because during training, models learn to generate responses based on statistical patterns in their training data, rather than being trained to verify factual accuracy. This is especially problematic in private and knowledge specific applications where the task-specific information was not part of the training data. And since the LLMs have been shown to store data in their parameters during training, alternative approaches have to be used to provide new, previously unavailable or updated information to the model, enabling us to improve the model’s capabilities without retraining.

These limitations introduce several significant problems in real-world applications. If an LLM is queried about information that emerged after its training cutoff, it might confidently try to present outdated, incorrect or even made-up information. In specialized domains, such as internal company documentation, technical specifications or internal tools, the models lack necessary context and may confidently generate plausible sounding but factually incorrect responses – a phenomenon known as hallucination. This is a big point of concern especially for users looking to utilize LLMs for internal purposes, or quality-sensitive uses, where one of the biggest hurdles is the lack of trust in the model’s responses. There is an option to train a private model on the company’s data, but this is often too resource-intensive and expensive to be considered viable.

Retrieval-Augmented Generation (RAG) addresses these limitations by combining the generative capabilities of LLMs with external knowledge retrieval systems. This architecture combines pre-trained parametric memory (the LLM) and non-parametric memory (an external, managed knowledge base) to generate responses that are grounded in the managed data. This approach is particularly valuable for applications requiring precise, domain specific, or private information where accuracy and traceability are essential.

A managed knowledge base provides a structured, persistent repository, that can be updated, versioned and queried with full traceability to source documents. These systems integrate mechanisms for document ingestion, metadata management, and controlled updates, all features essential for maintaining high data quality. However, for the model to leverage this information, it must be able to correctly interpret the retrieved data. There-

fore it is also very important to keep the high quality of comprehensive and structured data in the knowledge base. RAG systems depend on the LLM’s few-shot or one-shot learning capabilities: provided with external information, the LLM integrates this context into its generation, generating the final response from the supplied data rather than relying exclusively on what is stored in its parameters.

This thesis designs and implements a RAG application that operates over a managed knowledge base. The application accepts new documents, indexes them for retrieval, and grounds LLM context in the retrieved context, improving factual accuracy and providing direct traceability to source documents.

The thesis itself is structured as follows. Chapter 2 explores the relevant technologies: Large Language Models, RAG architecture, text embeddings, vector databases, hybrid retrieval and prebuilt tools available for RAG. Chapter 3 evaluates specific contemporary technologies available, defines requirements, and presents the architectural design and technology choices. Chapter 4 describes the implementation covering the document ingestion pipeline and the whole query pipeline from the original query to a final, improved response from the LLM. Chapter 5 validates the functionality and efficiency of the implemented system by using automated tests and experimentation.

Chapter 2

Overview of the current state of technologies

This chapter provides an overview of the current technologies and tools that are relevant to building a RAG application over a managed knowledge base. It will focus on basic concepts and techniques currently relevant for implementing the proposed solution, also mentioning some adjacent technologies that can be used to alter or improve the performance of LLMs. This chapter describes Large Language Models and their capabilities, Retrieval-Augmented Generation architectures, vector databases and vector search technologies. It concludes with an analysis of requirements and a survey of technologies for the proposed application.

2.1 Large Language Models

Large Language Models, serve their purpose as a generation component in a RAG system. Understanding LLM architecture, capabilities and limitations is important for understanding the need for RAG, its purpose and architecture.

Large Language Models are neural network-based systems trained on large amounts of text data to predict the next token in a sequence. Notable examples that have demonstrated significant advances in this area include models such as GPT-3 and LLaMA [17], which demonstrate newly acquired capabilities, including reasoning, code generation and few-shot learning [2]. These capabilities emerged from scaling of the model size and training data, instead of being explicitly programmed.

Notable examples that have demonstrated significant advances in this area include models such as GPT-3 and LLaMA [17], which demonstrate emergent capabilities, including reasoning.

2.1.1 Transformer Architecture

The foundation of modern LLMs is the Transformer architecture, first proposed in **Attention is all you need**[18]. This 2017 paper introduces a transformer block, based on attention mechanisms which allow the model to weigh the importance of different parts of the input when generating each output token.

Here the attention function is explained as a function mapping a query and a set of key-value pairs to an output. Query, keys and values are here transformed into vectors and an output is computed as a weighted sum of the values.

Modern LLMs typically use decoder-only architectures (as in GPT models) or encoder-decoder architectures (as in T5 or BART). Decoder-only models are autoregressive, generating text token by token. Recently they have shown to be highly capable in both understanding and generation tasks [14]. They are popular for their architectural simplicity and training efficiency. On the other hand, encoder-decoder models separate their input understanding from output generation. The encoder uses fully-visible masking in the encoder and the encoder-decoder attention, allowing each input token to engage with all other input tokens, all the while the decoder applies causal masking to prevent access to future outputs [12]. This design enables richer input representation, but at the cost of increased architectural, computational and memory requirements.

2.1.2 Techniques for guiding LLM behavior

While most LLMs are powerful from their training, their behavior can be significantly influenced and improved without the need to modify their weights or retraining.

Prompt Engineering

Prompt engineering is the practice of designing the prompts (input text) to cause the model to behave in a desired way. Common strategies include:

- **Zero-shot prompting:** Provide a task description without examples, rely on the model’s pre-training knowledge
- **Few-shot prompting:** Provide a small number of input-output examples directly in the prompt to illustrate the desired behavior
- **Chain-of-thought prompting** [19]: Encourage the model to show its reasoning process step-by-step, forcing it to reason about the task, improves performance on more complex tasks, requiring reasoning.
- **Instruction prompting:** Using precise instructions to guide the model’s behavior, defining the expected format, content, or style of the response

System Prompts and Role Instructions

Many LLM APIs support system prompts which set the context, role and the behavior rules for the model throughout the conversation. System prompts are useful for:

- Defining the model’s role or area of expertise for the given task – this enables specification of the model’s response style and content. Specifying these attributes more closely can also help make responses more concise by allowing it to be more narrow in its answer.
- Setting the constraints on the output format (JSON, structured text) – this allows the model to be integrated into other systems, workflows, automations, work with APIs, etc. It is useful when the output needs to be predictable, structured and consistent.

Fine-tuning and Parameter-Efficient Methods

Fine tuning works by adapting the model’s internal weights to fit better a specific task. However, full fine-tuning of a large language model is computationally expensive, time consuming and memory demanding, making it impractical for most applications.

To address these limitations, **parameter-efficient fine-tuning (PEFT)** methods have been developed. These work by introducing a relatively small number of additional trainable parameters, while not modifying the original model’s weights. These parameters are integrated directly into components of the transformer architecture and modify the model’s behavior while removing the need for full fine-tuning if the task requires slightly different but consistent behavior ¹.

Common PEFT approaches include:

- **Prompt-based methods (prefix tuning, soft prompts):** Instead of a prompt describing a scenario or providing an example of the desired behavior, soft prompting methods add learnable parameters to the input embeddings, that are optimized to perform better at the given task while keeping the original model’s behavior intact.
- **LoRA (Low-Rank Adaptation):** Injects trainable low-rank matrices into existing weight matrices, enabling efficient adaptation of the model – while keeping the original model intact.
- **IA³:** Multiplying the original model’s activations by three learned vectors results in a method that is even more parameter-efficient than LoRA by using vectors instead of matrices. These vectors scale transitional activations within the transformer.
- **QLoRA (Quantized LoRA), AdaLoRa(Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning), DoRA (Weight-Decomposed Low-Rank Adaptation):** Extensions of the LoRA method that improve hardware efficiency, dynamically allocate adaptation capacity, or improve the training efficiency.

In contrast to Retrieval-augmented generation which serves to provide external information to the model, PEFT methods instead modify the model’s internal weights to better fit the task. They help with defining the output style, its content and structure. In RAG systems, PEFT methods are optional, but useful for improving task execution and output quality.

Tool Use and Function Calling

Modern LLMs support function calling or the use of tools to perform external tasks they are not able to do by themselves. (API calls, code execution, calculations, search operations) If implemented correctly, this can significantly improve the model’s capabilities in the applications the tools are used for.

2.1.3 Limitations of LLMs

Despite the great advancements of the capabilities of modern LLMs, there are some fundamental limitations that persist in present-day models. These limitations originate from the nature of the model, its training process and the general-purpose nature of the models and cannot be avoided by scaling or modifying the model size, weights, or training data. Main limitations in this area are:

¹<https://huggingface.co/docs/peft/index>

Hallucinations

LLMs are sometimes prone to generating believable outputs that are factually incorrect or misleading – a phenomenon known as **hallucination**. This usually happens if the model has insufficient data to answer the given query and yet it attempts to answer it. The probabilistic nature of autoregressive language models is not concerned with factual accuracy in itself, it instead focuses on maximizing the likelihood of the next token in the sequence. The problem is also highlighted by the fact that the training procedures for such models often encourage them to produce an answer, even when there is low probability of the answer being correct. The behavior where the model indicates that it cannot reliably answer the question, in the situation where it evaluates that it is unable to answer reliably, has to be either explicitly programmed or introduced by prompt engineering.

Knowledge Cutoff

LLMs have a fixed **knowledge cutoff** which is based on the date of their training data’s end date. This means that once it is trained, the model cannot know about events or information that happened after this date. This makes it unsuitable for applications requiring up-to-date knowledge.

Context Window Constraints

While the context windows have grown rapidly in size, they are and always will be finite. This limits the amount of information that can be provided as context in a single request, making it a challenge to include extensive knowledge bases directly in prompts.

2.2 Retrieval–Augmented Generation (RAG)

This section introduces Retrieval–Augmented Generation (RAG), the core technology of this thesis that allows LLMs to leverage external knowledge to generate more accurate, reliable and traceable outputs. Furthermore, the key motivations for RAG are explored and explain the architecture of the system, including the role of text embeddings, chunking, vector databases and retrieval strategies.

2.2.1 Motivation for RAG

As mentioned in the previous section, standalone LLMs face basic limitations when dealing with specific knowledge intensive tasks. Large amount of these limitations originate from the reliance on parametric memory of the models, which is limited by the content of the training data [9]. RAG addresses this by combining the generative power of an LLM with external knowledge retrieval system [7].

RAG is especially valuable for applications which require:

- **Up-to-date information:** Access to information newer than the model’s training cutoff date.
- **Domain-specific knowledge:** Information not well represented in general training data, such as specialized documentation, technical specifications, emerging technologies, information that has too few examples to be included in the model’s parameters in a meaningful way.

- **Private knowledge:** Private, proprietary or sensitive information which cannot be included in public training data.
- **Accuracy, Traceability:** The ability to directly cite the source of information, and provide a step-up in reliability over a regular LLM response.

2.2.2 RAG architecture

A typical RAG system is built from several components working together, the components of a typical RAG pipeline include:

1. **Query Processing:** The user’s question is processed.
2. **Query Embedding:** The query is converted into a dense vector representation using an embedding model.
3. **Retrieval:** The query embedding is used to search a vector database for semantically similar document chunks.
4. **Ranking:** Retrieved chunks are ranked by their relevance to the original query.
5. **Context assembly:** Top-ranked chunks are selected, filtered and appended into a single context string optionally with metadata describing their location in the source documents.
6. **Generation:** The questioned LLM generates a response using the created context and the original query

Figure 2.1 summarizes this pipeline, showing how query processing, embedding, retrieval, ranking, context assembly and generation are connected in a typical RAG workflow. The original RAG paper by Lewis et al.[7] introduced this architecture in the context of question answering, demonstrating significant improvements over pure language models on knowledge-intensive tasks [15].

2.2.3 Text Embeddings and Similarity

Text embeddings are dense vector representations that store semantic meaning of the text. Similar texts should therefore have similar embeddings, enabling semantic search.

Sentence Embeddings

Modern embedding models, such as Sentence-BERT [13] and universal sentence encoder [3], are trained to produce embeddings that can be compared using cosine similarity efficiently.

Sentence-BERT, for example, uses siamese and triplet networks to derive semantically meaningful embeddings of sentences.

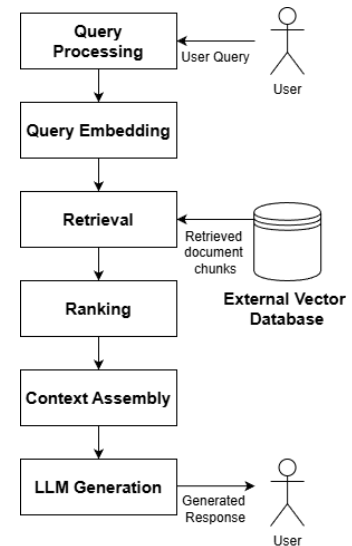


Figure 2.1: RAG pipeline architecture.

Embedding Models

Currently, there is a diverse range of embedding models available, which after emerging from the foundational work of word embeddings (Word2Vec [10], GloVe [11]), have evolved into more complex models that can capture the meaning of sentences and paragraphs. These models are typically divided into two categories by their architecture and training objectives:

- **Transformer-based models:** These models, including Sentence-BERT and MiniLM, are based on the transformer architecture and are trained to produce semantically meaningful embeddings of sentences and paragraphs. They tend to be more accurate but computationally expensive to run.
- **Non-transformer models:** These include models like FastText [1] and GloVe, which are based on simpler architectures. These models often produce less accurate embeddings, but are much faster.

Distance metrics

Similarity between embeddings is typically measured using:

- **Cosine similarity:** Measures the angle between vectors which are normalized by their magnitudes. This is the most common metric for the text embeddings.
- **Euclidean distance:** Measures the straight-line distance between vectors. Less commonly used due to its insensitivity to vector magnitude.
- **Dot product:** The inner product of vectors. Used when vectors are normalized (equivalent to cosine similarity).

For normalized embeddings, cosine similarity is preferred as it focuses on direction (semantic context) rather than magnitude.

2.2.4 Chunking Strategies

Documents must be split into **chunks** before embedding, as embedding models have maximum input lengths. Chunking strategies include:

- **Fixed-size chunks:** Simplest splitting of the input text by a fixed number of tokens or characters, can include optional overlap to preserve context.
- **Semantic chunking:** Splitting at natural boundaries of the document such as paragraphs or sections to try and preserve semantic coherency.

Chunk size is a critical parameter in a RAG system: too small, and the context can be lost due to fragmentation, too long, and the retrieval precision is reduced.

Recursive Character-Based Chunking

A middle ground between fixed-size and semantic chunking is recursive character-based splitting. This strategy attempts to split the document using a hierarchy of separators, starting with the most semantically relevant paragraph breaks (`\n\n`), and then continuing with line breaks (`\n`), sentence-ending punctuation (`. ? !`), spaces, and finally with character-sized breaks.

The algorithm operates recursively: It first tries to split the input at the highest level separator, if any resulting chunk still exceeds the maximum chunk size, it recurses on that chunk with the next separator until all chunks are within the size limit. This method preserves semantic structure the best, all the while making sure that the chunks are close to the specified size constraint.

Furthermore, optional overlap between neighboring chunks can be added in a second pass. A configurable amount of tokens from each chunk can be added in front of the next chunk, this helps to preserve context in-between chunks, improving retrieval performance.

Typical chunk sizes for RAG system range from 128 to 512 tokens (250–1000 characters) with a 10–20% overlap, with the optimal chunk size depending on the specific use case. Generally smaller chunks improve factual retrieval, while larger chunk sizes are useful when improving contextual understanding is more important.

2.2.5 Document Ingestion and Metadata management

Before chunking and embedding, a RAG pipeline must extract raw text from the source documents. Different formats naturally require different extraction methods, ranging from simple UTF-8 text extraction to more complex parsing of PDFs due to their specific structures. Text has to be appropriately normalized, cleaned and preprocessed to prepare it for chunking and embedding.

2.2.6 Vector Databases

Vector databases are a type of database that is used to store and retrieve embeddings of data in a vector space. They are optimized for semantic similarity search over high-dimensional vectors. Along with the embeddings they store original text chunks, metadata document ID, timestamps, etc.

Storage Model

A vector database entry typically contains:

- **Vector ID:** Unique identifier for the embedding.
- **Embedding vector:** The dense vector semantic representation.
- **Payload:** Associated data (text chunk, document ID, metadata, etc...).

This structure enables retrieval of most similar chunks of text based on their meaning rather than exact/partial matching.

Indexing Structures

Vector databases use specialized index structures to enable fast approximate nearest neighbor (ANN) search:

- **HNSW (Hierarchical Navigable Small World)** [8]: A graph based structure that builds a multi-layer graph where each layer is a subset of the previous layer. HNSW provides excellent query speed and recall, making it popular choice for RAG systems.

- **IVF (Inverted File Index)** [5]: Partitions the vector space into clusters (Voronoi cells) and restricts the search to the `nprobe` nearest cells. Used in systems like FAISS which implements it efficiently on GPU’s [6].
- **PQ (Product Quantization)**: Compresses vectors using quantization, reducing memory usage at cost of accuracy. Typically combined with IVF, to create IVF-PQ index, which is particularly useful for large datasets.

HNSW and IVF-PQ represent two popular, but different approaches to ANN search, based on graph traversal and quantization respectively.

These structures trade off between query speed, memory usage, retrieval accuracy. HNSW is the preferred option for RAG applications for its high recall and lower query latency, particularly in CPU-based systems, while IVF-PQ has better memory efficiency and is preferred for larger-scale deployments.

2.2.7 Hybrid Retrieval

Hybrid retrieval strategies combine sparse and dense (vector based) retrieval to reduce each others limitations, such as lexical mismatch in sparse methods and semantic overgeneralization in vector based methods. This improves the retrieval quality across a wider range of queries and is often used in RAG systems. In practice, hybrid retrieval typically combines BM25 (Best-Match 25, a sparse retrieval method) with vector-based similarity search using one of the following strategies:

- **Score fusion**: Combines BM25 and vector similarity scores using a weighted average or reciprocal rank fusion.
- **Re-ranking**: A layered approach where BM25 retrieves a set of candidate chunks to be re-ranked by a vector similarity model.
- **Cross-encoder re-ranking**: Utilizing a more computationally expensive but accurate model to re-rank candidates retrieved by both methods using the combined scores.

Evaluations of hybrid retrieval on the BEIR benchmark [16] show that it often outperforms purely dense or sparse retrieval methods, providing a more robust retrieval performance across a wide range of queries.

Dense retrieval: Vector search

Vector search is a dense retrieval method that uses the semantic meaning of the query text to find relevant documents. It is useful for queries that do not have exact keyword matches in the documents but are semantically related. Vector search is effective for:

- Synonyms and paraphrases.
- Queries for contexts rather than exact terms.
- Queries with unclear or vague language.

Lexical search: BM25

BM25 is a sparse, term-based retrieval model which ranks documents based on exact token matches using term frequency and inverse document frequency weights. In contrast to vector search, BM25 does not have any semantic understanding of the text, but it is effective for exact keyword matching instead. BM25 is very effective for:

- Technical terms and proper nouns.
- Queries with specific keywords.
- Cases where exact term matching is important.

Full-Text Search with SQLite FTS5

BM25 scoring commonly associated with dedicated search engines. However, for smaller datasets, the full-text search capabilities of SQLite’s FTS5 extension² can be a practical alternative, especially for applications that already use SQLite for metadata storage. FTS5 provides efficient full-text search through virtual tables that store an inverted index over textual data. It supports `MATCH` queries (FTS5 specific syntax) for full-text filtering and allows relevance ranking using the built-in `bm25()` function directly in SQL queries. This reduces system complexity by avoiding the need for a separate search engine such as Elasticsearch or OpenSearch. The trade-off is that, although SQLite FTS5 is reasonably efficient for small to medium-sized applications, it lacks the scalability and advanced features of dedicated search engines, which can be a critical requirement for larger RAG applications.

Reciprocal Rank Fusion (RRF)

Reciprocal Rank Fusion (RRF) [4] is a rank-based score fusion method that sorts the retrieved documents using a naive scoring formula. For a document d from a set of D documents that are ranked using a set of rankings R of which each is a permutation of the documents in D , the RRF score is computed as:

$$\text{RRF score}(d \in D) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)} \quad (2.1)$$

Where k is a constant (in the paper intuitively selected as 60) which helps to bring out the lower-ranked documents which would otherwise be overlooked even while being relevant and $\text{rank}_r(d)$ is the rank of document d in ranking r .

The main advantage of RRF over other weighted score fusion methods is that it operates purely on ranks of the documents rather than their raw scores. In simpler terms, RRF gives more weight to documents that are ranked higher in any of the input rankings. This approach has been shown to outperform individual ranking as well as other, more complex approaches such as Condorcet voting and CombMNZ [4].

2.3 Tools and Abstraction Layers

There have been several frameworks and tools developed to simplify the development of RAG applications. In this section, some of the most relevant ones are briefly explored.

²<https://www.sqlite.org/fts5.html>

2.3.1 RAG Frameworks

Frameworks such as **LangChain**³, **RAGFlow**⁴, **LLamaIndex**⁵ and **Haystack**⁶ provide pre-built components for document ingestions, chunking, embedding, vector storage and retrieval, as well as integration with LLM APIs.

LangChain offers a wide set of integrations and supports both regular sequence based and agent-based applications. **RAGFlow** is a RAG framework that provides a user-friendly interface for building and deploying RAG applications along with support for locally deployed LLMs. **LLamaIndex** specializes in document indexing and retrieval customization more than other frameworks. **Haystack** offers a powerful modular and pipeline-oriented framework with built-in REST API support.

While these frameworks provide a strong foundation for building RAG applications, they, like any abstraction layer, can introduce additional overhead and limit control over the pipeline. On the other hand, a custom implementation provides full control over each pipeline component, with the inherent trade-offs of such an approach — increased development time and implementation complexity.

2.3.2 LLM Abstraction Layers

LLM abstraction layers such as **LiteLLM**⁷ provide a unified API for calling multiple LLM and embedding model providers (OpenAI, Anthropic, local models via Ollama, and others). This allows switching between providers with minimal code changes, enabling seamless switching of different providers and models for testing, optimization, cost management, and long-term maintainability, while also making the system easier to adapt when newer or better-performing models become available in the future.

³<https://python.langchain.com/docs/>

⁴<https://ragflow.io/docs/>

⁵<https://docs.llamaindex.ai/>

⁶<https://docs.haystack.deepset.ai/>

⁷<https://docs.litellm.ai/>

Chapter 3

Design and Technology Selection

This chapter evaluates the current state of technologies and tools described in the previous chapter, defines the requirements for the proposed RAG application, selects the most suitable technologies for each component of the proposed system and describes the architecture that emerges from these choices and requirements. The goal is to reach a complete design for RAG system that can serve as a foundation in the following implementation chapter.

3.1 Critical Evaluation of Existing Approaches

The technologies described in the previous chapter provide a wide range of possible approaches to building a RAG system. This section evaluates the individual technologies for their suitability, performance, implementation difficulty and trade-offs.

RAG versus fine-tuning. Fine-tuning and PEFT methods (Section 2.1.2) modify the model’s internal weights to improve performance in a specific task. However, they require retraining whenever there is a need to update the model’s knowledge, making them impractical for applications where documents are added or updated frequently. RAG, on the other hand, provides a conceptually straightforward way to incorporate additional information by separating the knowledge storage from the language model. Data stored this way can be updated, versioned and managed independently of the model. This approach also enables traceability and source attribution, where the facts in the generated response can be directly linked to their respective source documents – a feature not possible with any model-based approach.

Custom implementation versus RAG frameworks. Frameworks such as LangChain and Llamaindex (Section 2.3) provide pre-built components and even the whole pre-made pipelines for RAG applications. While they can serve as a great starting point for building such applications, they can abstract away important details and introduce additional hidden complexity. A custom implementation, while being more time consuming and complex to develop, provides full control over each component of the pipeline. And while it may be difficult to implement on a level of a professional commercial solution, it introduces transparency and a deeper understanding of the underlying technologies, which is one of the main goals of this thesis.

Hybrid retrieval versus vector or sparse retrieval alone. Evaluations on the BEIR benchmark [16] demonstrate that hybrid retrieval consistently outperforms both purely dense and purely sparse methods. Dense retrieval captures semantic similarity but

misses exact keyword matches, while sparse retrieval does the opposite. A combination of both of these methods using a method such as Reciprocal Rank Fusion provides improved retrieval performance across a wider range of queries.

3.2 Requirements for the Proposed Application

Based on the evaluation above, RAG architecture specifics, and UX considerations, the following requirement specification for the proposed RAG application is defined:

3.2.1 Functional Requirements

1. **Document upload and ingestion:** Accept documents in PDF, plain text, Mark-down and optionally other formats, Extract text, split into chunks, generate embeddings and index them for retrieval. Ingestion should ideally run asynchronously to avoid blocking the main application and provide feedback on the ingestion process.
2. **Three retrieval modes:** Support hybrid retrieval by definition includes combining both dense (vector) and sparse (syntax-based) methods of retrieval. For this reason, exposing and testing these methods separately is also important. Therefore the application should, at least for validation and testing purposes, support all three retrieval modes – dense, sparse and hybrid.
3. **Answer generation with source attribution:** Use of an LLM to generate answers with retrieved context. The response itself should include the retrieved context, and link to specific document chunks and/or their location in the source documents.
4. **Job tracking:** Provide visibility into the status of document ingestion, retrieval and generation jobs using error messages.
5. **Web-based user interface:** Provide a FastAPI-based REST API and a simple web-based user interface for document upload, question submission, response display and optional debugging information such as retrieved chunks, inspection of the constructed prompt without the LLM call itself, optional health endpoints, etc.

3.2.2 Non-Functional Requirements

1. **Local-first deployment:** The application should run entirely on a single machine using Docker Compose for infrastructure and a local application server.
2. **Flexible architecture:** Embedding provider and LLM provider should be replaceable through configuration.
3. **Environment-based configuration:** All settings should be configurable via environment variables.
4. **Transparent data storage:** For the purposes of testing, all data in the knowledge base should be visible and accessible in a human-readable format.

3.3 System Architecture Design

3.3.1 High-Level Architecture

The main application follows an intuitive three-tier architecture:

- **API layer:** A FastAPI based REST API that acts as the main entry point for all of the communication with internal components of the system.
- **Business logic layer:** Modules for the internal logic of the application including document ingestion, query processing, retrieval, ranking and answer generation.
- **Storage layer:** for this purpose the application uses two different storage systems: `Qdrant`¹ as a separate docker container for dense embeddings, `SQLite with FTS5`² for document metadata, chunk content storage and sparse retrieval.

Note: LLM and embedding API calls are routed through LiteLLM³, this happens in the business logic layer and is separated from the rest of the logic. This external dependency provides an unified interface for multiple providers in case of a need to switch providers. A complete overview of these layers and their interactions is shown in Figure 3.1.

3.3.2 Module Structure

The application based on the requirements described above should follow a modular structure, with the following main modules:

- `ingestion/` – Text parsing and extraction (PDF, plain text, markdown), chunking and the indexing pipeline.
- `storage/` – SQLite repository for documents, chunks and jobs. Qdrant storage adapter for vector storage and retrieval.
- `retrieval/` – Dense, sparse and hybrid retrievers implementation.
- `generation/` – Prompt construction and LLM answer generation logic.
- `embeddings/` – Embedding client adapter for external embedding providers.
- `core/` – Application core logic, settings, configuration.
- `api/` – FastAPI route handling organized by endpoint group (documents, queries, jobs, health).
- `ui/` – A simple static HTML/CSS/JS frontend served by the application. Handling interaction with document upload pipeline and question submission and response pipeline respectively.

¹<https://qdrant.tech/documentation/>

²<https://www.sqlite.org/fts5.html>

³<https://docs.litellm.ai/>

3.3.3 Data Flow

The application has two primary data flows:

Ingestion pipeline: When a document is uploaded, the API endpoint saves the file and creates a new ingestion job. This job is asynchronous to the main query pipeline and processes the document by saving it into the storage and creating its metadata records in the SQLite database. The text is then extracted from the document, normalized and split into chunks using recursive character chunker, the chunks are then embedded using an external embedding provider. Chunks are then stored into SQLite with their metadata and the final embeddings are stored in Qdrant with a reference to the chunk ID in SQLite.

Query pipeline: When a query which requires dense retrieval is submitted, the application processes the query by first embedding it using the same embedding provider as for the chunks (This has to be done with the same provider, because the embeddings need to be aligned in the same vector space). The query embedding is then used to search the Qdrant vector database for similar chunks, which are then retrieved and ranked. In the case of a need for sparse retrieval, the difference is that the query is instead processed using a BM25 search over the SQLite FTS5 virtual table, which returns the relevant chunks based on exact token matches. Both of these methods return top- k chunks with their content and metadata, which in the hybrid retrieval mode combine the results using Reciprocal Rank Fusion. Retrieved chunks are then assembled into an unified prompt built from the original query, retrieved context and citation instructions. Finally prompt is then sent to the LLM for answer generation and then the response is returned to the user along with the retrieved context.

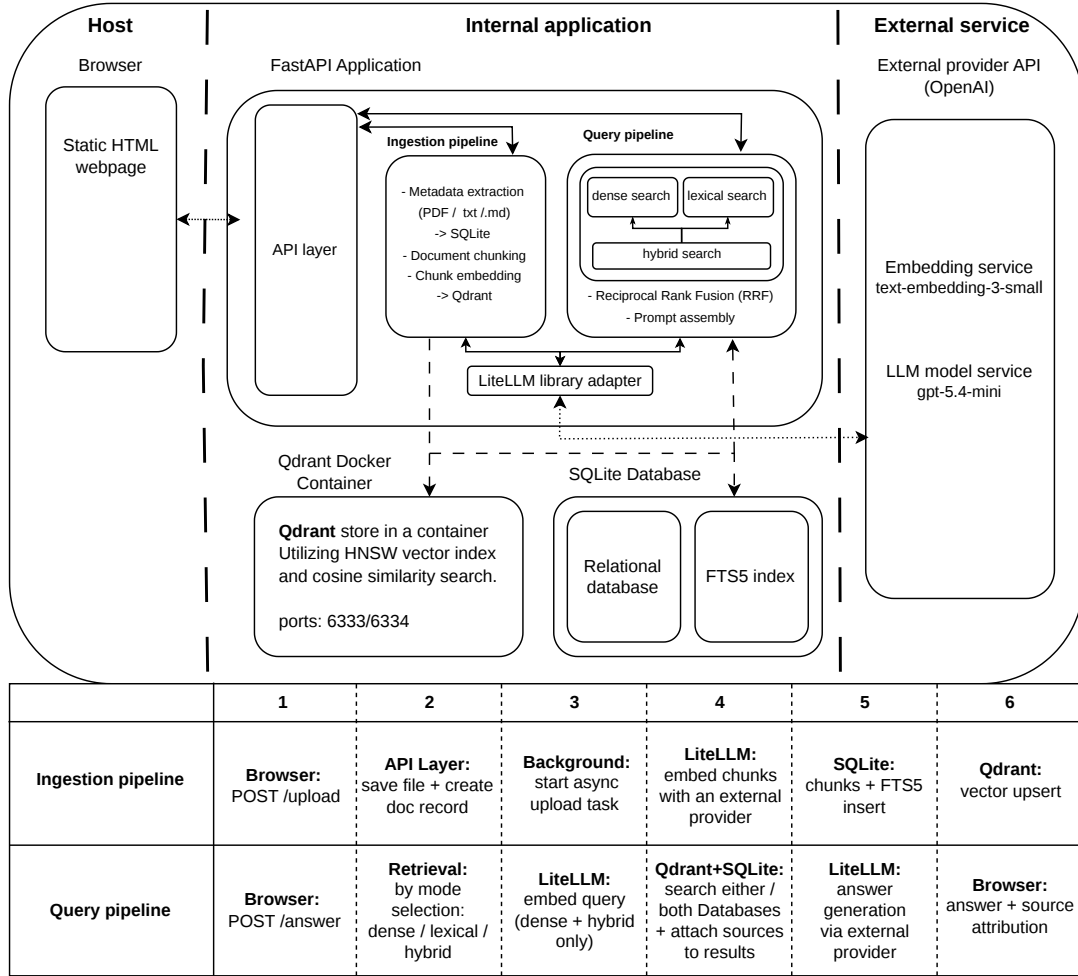


Figure 3.1: High-level system architecture. The FastAPI application communicates with Qdrant (running in Docker) for vector storage, SQLite for metadata and lexical search, and external LLM/embedding APIs through LiteLLM.

3.4 Technology Selection and Justification

This section summarizes the specific technologies chosen for the implementation. The selection is based on the requirements presented in the requirements section 3.2 and the technology evaluation in section 2.

3.4.1 Programming Language and Framework

Python programming language is chosen as the implementation language in this project due to its dominance in the field of machine learning and natural language processing, as well as an extensive amount of libraries for embeddings, LLM interaction and vector database clients which are all Python-native.

FastAPI⁴ is selected as the web framework for its native async support, automatic OpenAPI documentation generation and integration with Pydantic for request/response validation. Uvicorn serves as the ASGI server. The uv package manager is used for fast and reproducible dependency resolution.

3.4.2 Vector Database

Qdrant⁵ is selected as the vector database because it provides a practical balance between functionality and simplicity of deployment. It offers a ready-to-use server deployment solution with persistent storage via Docker. There is also an efficient similarity search for embeddings along with metadata based filtering. The main advantages include a clear API, aforementioned support for both filtering and vector search, and a modern architecture designed for vector retrieval tasks.

Compared to the alternatives also considered:

- **FAISS** [6] is a library and lacks built-in data persistence, requires custom code for data management. Better suited for prototyping on GPU's.
- **Chroma** is simple and a bit more crude, also offers a fewer selection of features for filtering.
- **Milvus** provides distributed capabilities that introduce unnecessary complexity for a small scale deployment.

3.4.3 Lexical search and Metadata storage

SQLite is the architectural choice for metadata storage (documents, chunks, indexing jobs) due to its easy configuration and a straightforward file-based persistence. SQLite's FTS5 extension⁶ provides BM25 based lexical search through virtual tables with no need for a separate search engine. This minimalizes overhead and simplifies the infrastructure – only Qdrant requires Docker and the rest of the storage is a single `.db` file.

3.4.4 Embedding and Generation Models

OpenAI's **text-embedding-3-small** model⁷ is selected for embeddings, producing 1536-dimensional vectors with a maximum input of 8191 tokens. It offers a practical balance between quality and a low-cost requirement imposed by the nature of the project. According to the provider, when compared to older models from the same provider it performs better on benchmarks and is radically cheaper. More advanced alternatives, such as **text-embedding-3-large**, provide higher accuracy at the cost of increased dimensionality (2x – 3072) and higher price that doesn't justify it for this use-case. Open-source/local models (Cohere, Mistral, or BGE families) offer comparable performance but require extra effort to implement and deploy.

For answer generation, OpenAI's **gpt-5.4-mini** is used as the configured LLM in the final prototype. The original implementation plan considered **gpt-4o-mini**, but **gpt-5.4-mini**

⁴<https://fastapi.tiangolo.com/>

⁵<https://qdrant.tech/documentation/>

⁶<https://www.sqlite.org/fts5.html>

⁷<https://platform.openai.com/docs/guides/embeddings>

was selected during final implementation for stronger instruction following in this prototype. The exact model is not a fixed architectural requirement, but rather an example of the attributes the final model should provide.

Both embedding and generation calls are routed through **LiteLLM**⁸, which brings in an easy provider switching and a potential for easy testing with various other embedding and LLM models, including local models.

3.4.5 Chunking Strategy Parameters

Recursive character-based chunking is chosen with the following default parameters:

- **Chunk size:** 1000 characters – fits within embedding model’s token limit while having enough context per chunk for a
- **Overlap:** 150 characters (15%) – to preserve context across neighboring chunks.
- **Separator cascade:** Paragraph breaks, line breaks, sentence-ending punctuation, spaces and hard character split.

These parameters are configurable via environment variables, allowing experimentation without extensive code changes.

3.4.6 Retrieval Strategy

The final application supports three retrieval modes:

- **Dense:** Cosine similarity search over Qdrant embeddings.
- **Lexical:** BM25 scoring utilizing SQLite FTS5 MATCH queries.
- **Hybrid:** Independent dense and lexical retrieval fused with Reciprocal Rank Fusion (Equation 2.1) using $k = 60$ [4].

Hybrid mode is the recommended default, as it is a combination of semantic and lexical matching, producing best results by combining the strengths of both approaches. Each retriever contributes top_k candidates before fusion. Then the results are combined by their ranking into one combined result.

3.5 API Design

The proposed application should expose the following REST endpoints:

⁸<https://docs.litellm.ai/>

Method	Path	Description
POST	/api/documents/upload	Upload a document
GET	/api/documents	List all documents with their metadata
POST	/api/query/dense	Dense retrieval selector
POST	/api/query/lexical	Lexical retrieval selector
POST	/api/query/hybrid	Hybrid retrieval (RRF) selector
POST	/api/query/answer	Retrieval + LLM answer generation
POST	/api/query/prompt-debug	Build prompt without LLM call
GET	/api/jobs	List indexing jobs with status
GET	/api/health	System health check
GET	/	Serve frontend UI

Table 3.1: Planned API endpoints.

Query endpoints accept a JSON body with a `query` string, optional `top_k` parameter, and flags for retrieval methods.

3.6 Testing and Evaluation Strategy

The testing strategy is designed to verify the proper functionality at each stage development, without requiring the full working application and functional API first.

Unit tests cover individual components in isolation: the chunking algorithm (correctness of splits, overlap, ordering), text extraction (format handling, fail cases), embedding adapter (adherence to protocol), prompt building & generation response (template quality and response testing)

Integration tests Validate the API endpoints functionality using FastAPI’s `TestClient`, which runs the application without creating a real HTTP and socket connection, communicating directly with the FastAPI code.⁹ These tests are useful for full request–response testing cycle.

Evaluation approaches Formal evaluation using standardized benchmarks is ideal but not expected due to its complexity. The retrieval system is evaluated by comparing the quality of retrieval modes with a series of different user queries from a catalogue of questions which test the retrieval capabilities of different search modes. The basic idea between these different tests is:

- **Dense retrieval test cases:** The query is phrased in a way where there isn’t a big possibility of a lexical match, but the meaning it conveys is the same as a specific part of the knowledge base.
- **Sparse retrieval test cases:** The query contains keywords from the knowledge base while its meaning has nothing to do with at least the given retrieved chunk.
- **Hybrid retrieval cases:** queries combine semantic intent with important keywords, testing whether fusion improves the final retrieved context.

⁹<https://fastapi.tiangolo.com/>

Chapter 4

Implementation

This chapter describes the implementation of the RAG application designed in Chapter 3. It presents the project structure, the key modules and their interactions, important implementation decisions and deviations from the original design. The focus is on important and distinct features of this implementation, rather than a detailed review of the codebase.

4.1 Project Structure and Configuration

The project is implemented as a Python project with a `src` layout and dependencies managed by `uv`. Its main structure is:

```
src/app/
  core/          # Configuration
  ingestion/     # Extractors, chunker, indexing pipeline
  storage/       # SQLite repositories, Qdrant store adapter
  retrieval/     # Dense, lexical, hybrid retrievers
  generation/   # Prompt builder, answer generator
  embeddings/    # Embedding adapter, providers
  api/          # FastAPI route handlers
  ui/           # Static HTML frontend
  main.py       # Main application entrypoint
tests/         # Unit and integration tests
pyproject.toml # uv project metadata and dependency definitions
docker-compose.yml # Local Qdrant service
Makefile       # Local development and test commands
```

4.1.1 Configuration Management

Application configuration is managed through a `Settings` class based on `pydantic-settings`' `BaseSettings` ¹. This class loads values from local `.env` file local to the repository and from environment variables, and validates them when the settings object is created. The validation checks include a non-empty Qdrant URL using `http://` or `https://`, vector-size values, positive chunk size, non-negative chunk overlap smaller than the chunk size, and non-empty critical strings such as model names and database paths. When API-based

¹https://docs.pydantic.dev/latest/concepts/pydantic_settings/

embeddings are enabled, `PROVIDER_API_KEY` must also be set. Invalid configuration raises a validation error before the application is fully initialized.

4.1.2 Deployment Setup

For local execution, the FastAPI application starts on the host by `make app-run`, which runs Uvicorn with auto-reload enabled. This setup is useful during implementation so that code changes were reflected automatically.

A separate infrastructural element for Qdrant service (`qdrant/qdrant:latest`) was deployed with the use of Docker Compose. This container exposes the ports 6333 (HTTP API) and 6334 (gRPC - interface for faster production-tier searches) to the host. A named Docker volume (`qdrant_storage`) , stores Qdrant data persistently container restarts.

Makefile provides following targets:

- `make qdrant-up` to start the vector database
- `make qdrant-check` to check Qdrant connectivity
- `make app-run` to start the application server
- `make test` to run the `unittest` test collection

4.2 Document Ingestion Pipeline

The ingestion pipeline transforms uploaded documents into indexed and searchable chunks. It is the most complex subsystem of the application, involving text extraction, chunking, embedding and persistence in two separate databases.

4.2.1 Text Extraction

The `extract_text()` function forwards the file to the appropriate extractor based on its file extension. Plain text (`.txt`) and Markdown (`.md`) files are decoded as UTF-8. PDF files are processed using `pypdf`², which extracts the target text page by page and joins the results with newline separators.

After extraction, the text gets normalized: Windows-style line endings (`\r\n`) are converted into Unix-style (`\n`), trailing whitespace is stripped from each line along with extra trimming of the rest of the text.

4.2.2 Recursive Character Chunking

The chunking implementation follows the recursive character-based strategy described in Section 2.2.4, with a two-pass approach.

First pass – recursive splitting: The algorithm uses an ordered list of separators: `"\n\n"`, `"\n"`, `" "`, `"?"`, `"!"`, `" "`, and the empty string (hard split in between characters). The implementation first calculates a base size as `chunk_size_chars - chunk_overlap_chars`. Given input text, the splitter selects the first separator that appears in the text. Pieces that still exceed the base size are recursively split using smaller separators. Neighboring small pieces are merged up to the base size. Separators are kept attached to the previous segment so punctuation and newline context are preserved.

²<https://pypdf.readthedocs.io/>

Separators themselves are attached to prior text segment to preserve punctuation to each chunk.

Second pass – overlap application: After the recursive split creates a list of base chunks, overlap is applied sequentially. For each chunk after the first, a prefix consisting of the last n characters of the previous output chunk is attached before the chunk itself. This helps to preserve context at the borders of neighboring chunks, although it does not guarantee perfect semantic continuity.

Figure 4.1 illustrates this two-pass implementation, where recursive splitting by separators is followed by overlap insertion between neighboring chunks.

Recursive character splitting with separator cascade + overlap application

Input text: *BitSwan runs automations in notebooks.\n\nEach event flows through pipeline cells.\nCells process events from a Source to a Sink.*

Example settings: chunk_size = 58 characters
overlap = 15 characters
base_size = 58 - 15 = 43 characters

Separator hierarchy:
\\n\\n → \\n → " → . → ? → ! → " → " → ""

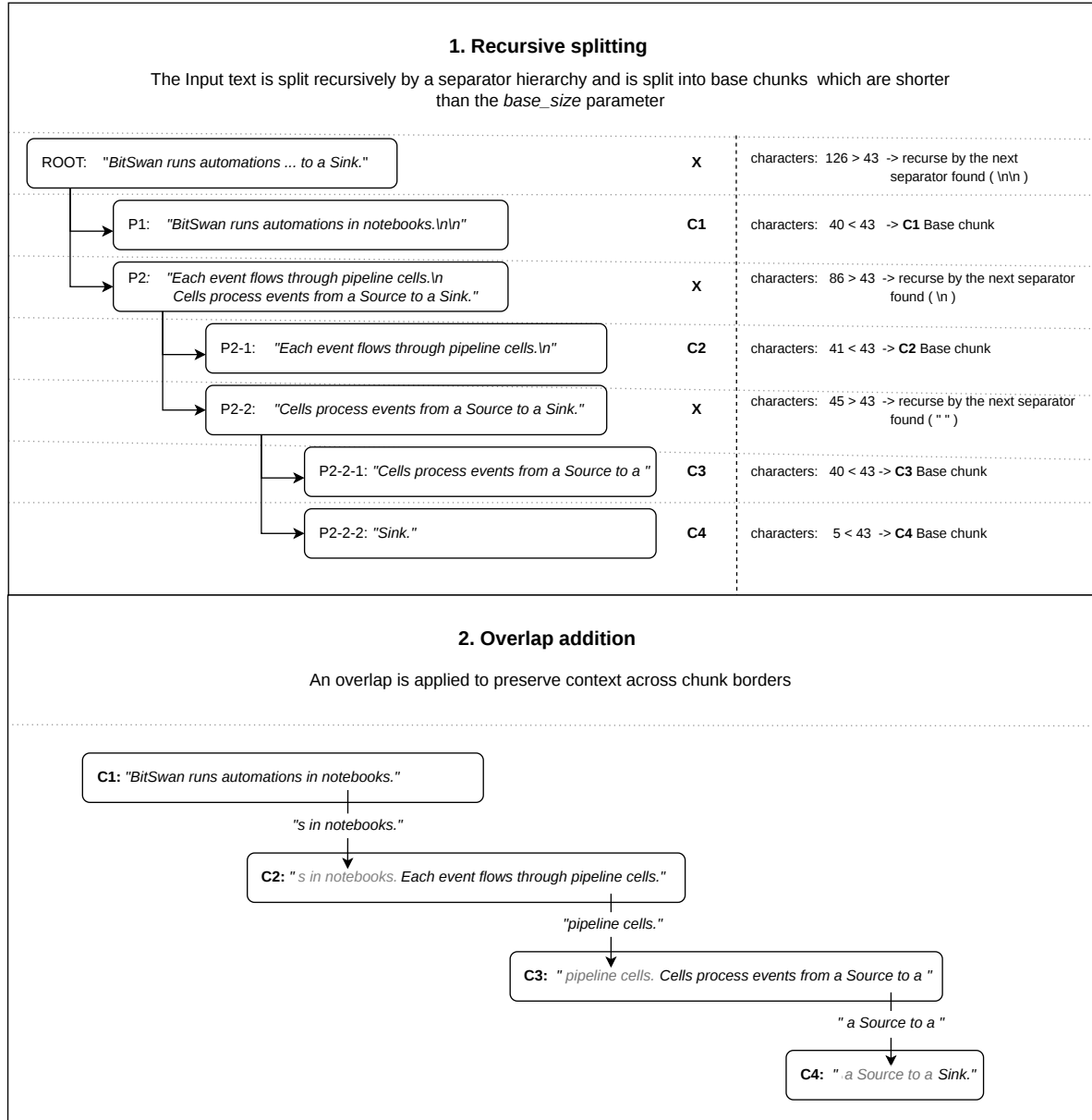


Figure 4.1: Recursive character chunking with separator cascade. Pass 1 splits the input recursively using the separator hierarchy until every piece fits within **base_size**. Pass 2 adds overlap by prefixing each chunk with the last **overlap** characters of the previous output chunk.

4.2.3 Embedding Generation

Embeddings are generated using an external embedding provider through the `EmbeddingClient` adapter. In the production implementation, the provider embedding client, calls `litellm.embedding()` with the configured model (default: `text-embedding-3-small`).

All chunks for a document are embedded in a single batch API call. The adapter takes note of the success and failure per the chunk via `EmbeddingItemResult` objects. In the case of a failure of the chunk embedding, the chunk is still stored in the SQLite database, but its `embedding_model` is `NULL` and no Qdrant vector is stored. Other chunks are not affected and the process continues, only if all embeddings fail, the whole ingestion job is marked as failed.

4.2.4 Dual-Store Persistence

After chunking and embedding, the results are saved into two separate storage systems with complementary roles: SQLite keeps the relational metadata, chunk content and the inverted index(FTS5) for lexical search, while Qdrant stores the vector embeddings for dense retrieval. The SQLite schema, initialised at application startup by `initialize_sqlite_schema()`, is summarised below:

```
documents (
    id, source_path, filename, source_type,
    size_bytes, checksum,
    status          -- pending | running | success | fail
)

chunks (
    id, document_id, chunk_index,
    content,
    embedding_model -- NULL when embedding failed for this chunk
)

chunks_fts (
    chunk_id, document_id, content
) -- FTS5 mirror used for BM25 lexical retrieval

jobs (
    id, job_type, document_id,
    status,          -- pending | running | success | fail
    payload_json,    -- embedding and Qdrant indexing statistics
    error_message
)
```

Figure 4.2: Most relevant SQLite values used by the application. The `documents` table stores source metadata and ingestion state; `chunks` stores retrievable text units and their embedding status; `chunks_fts` mirrors chunk text for BM25 lexical search; and `jobs` stores background indexing state, errors and summary statistics.

With this schema defined, the storage systems receive the following data:

- **SQLite:** Each chunk is inserted into the `chunks` with its content, document ID, chunk index and the embedding model name. At the same time the same chunk content is

inserted into the `chunks_fts` virtual table, which builds the inverted index for lexical search.

- **Qdrant:** Each successfully embedded chunk is inserted as a point with a unique, deterministic UUID5 identifier (generated from the chunk ID), the embedding vector, and a minimal set of metadata containing `chunk_id`, `document_id` and `chunk_index`.

The `replace_document_chunks()` function first removes the old chunks of a document and then adds new ones in a single step. This allows the document to be processed and updated without leaving the system in an inconsistent state. In Qdrant, the point IDs are generated with UUID5, so indexing the same chunk again will produce the same point ID. This makes the repeated ingestion of the same document consistent instead of creating duplicate chunks and embeddings. In the relational database, foreign keys on `chunks.document_id` and `jobs.document_id` preserve consistency. Qdrant does not support foreign keys, so it depends on storing the same document ID on each point and using it for filtering and deletion.

4.2.5 Background Task Execution - Ingestion Jobs

Indexing runs as a FastAPI background task, using the framework's built-in `BackgroundTasks` mechanism. When a document is uploaded, the API endpoint returns immediately with the document and job IDs, while the actual processing of the indexing pipeline runs asynchronously in the background.

The simple lifecycle of a job is:

`pending` → `running` → `success` or `fail`.

During normal execution, exceptions raised inside the indexing pipeline are caught and recorded as failed job records with an error message. Client can inspect the job endpoint to see the final status.

This approach was chosen for its simplicity compared to more complex solutions that use a message queue. Single-process background task is sufficient for a small-scale application, while still providing the benefits of asynchronous processing. The trade-off is that there is no retry mechanism and running jobs are lost if the process restarts.

4.3 Retrieval

The retrieval system is organized around a single `Retriever` protocol. This protocol defines one common method: `retrieve(query, top_k)`, implemented by `DenseRetriever`, `LexicalRetriever`, and `HybridRetriever`. The selected mode is passed to `build_retriever`, which constructs the selected implementation while keeping the rest of the application independent of the retrieval strategy. All retrievers return normalized `RetrievedChunk` objects containing the chunk content, document metadata, and a retrieval score.

The retrievers use the following methods for retrieval:

4.3.1 Dense Retrieval

The `DenseRetriever` embeds the query using the same embedding client used during indexing, then it searches the Qdrant collection for the most similar vectors using cosine similarity. Qdrant returns the top- k most similar points with their point IDs, similarity scores and payload data. Using this data, the retriever fetches the chunk content and the name

of the source document from SQLite, and returns the results as a list of `RetrievalResult` objects with the retrieved content, metadata and similarity scores.

4.3.2 Lexical Retrieval

The `LexicalRetriever` performs keyword based search over the indexed chunk text using SQLite FTS5. It first splits the query into separate searchable parts and builds an FTS5 MATCH query from them.

The retriever first attempts a stricter search where all query items must appear in the same chunk. This strong match doesn't always return results, so if the results are empty, it falls back to a weaker search where any of the terms can match. This keeps the default behavior for queries that have a strong lexical match, while still providing results for queries that don't have a strong match but still contain relevant keywords.

SQLite FTS5 ranks the matches with BM25. Since SQLite returns these scores as negative values, the retriever inverts them to positive values so that they are compatible with the scores from dense retrieval.

The retriever returns a list of `RetrievalResult` objects with the retrieved content, metadata and inverted BM25 scores.

4.3.3 Hybrid Retrieval with RRF

The `HybridRetriever` runs both the dense and lexical retrievers independently, each contributing top_k candidates. These results are then fused using Reciprocal Rank Fusion (Equation 2.1) to produce a single ranked list of results.

The fusion algorithm maintains a `_FusedChunkCandidate` for each unique chunk ID across both ranked lists. Each candidate accumulates a fused score and tracks the best rank across both lists. Incidental duplicates are removed based on the chunk ID, so if a chunk is retrieved by both methods, it only appears once in the final result. The final results are sorted by score descending and then by rank ascending.

Reciprocal Rank Fusion was chosen because of its tendency to produce better results than other fusion methods, such as CombSUM or CombMNZ. It prefers chunks that are ranked highly by both retrievers, while still allowing chunks returned by only one method to appear in the final list. This makes hybrid retrieval more robust to differences in score scales, as it relies on rank positions rather than raw scores, which are not directly comparable across retrieval methods.

4.4 Answer Generation

After retrieval produces a list of the most relevant chunks, the application needs to construct a prompt that the language model can use to produce the final answer. This part of the implementation documents how retrieved context is assembled into a prompt together with the original query, and what instructions are given to the LLM to produce the final answer with source attribution.

4.4.1 Prompt Construction

The prompt builder operates in two primary user-selectable modes, with and without context inclusion, with one additional internal fallback mode that is triggered automatically

when retriever does not return any results. This design choice was made to allow simplified testing and evaluation of retrieval quality. Here are the details of each mode:

1. **Grounded mode:** Used when retrieved chunks are available and context inclusion is enabled. The prompt includes an instruction to answer only from the provided context, cite sources inline using [S1], [S2] format, and indicate lack of information if the context is insufficient. Each source is formatted with its identifier, filename, document ID, chunk index and full content.
2. **Fallback no-context mode:** This is an internal mode that is not selected directly by the user, it is triggered automatically when grounded mode is enabled but retrieval returns no relevant results either by error or lack of relevant information in the knowledge base. Here, instead of sending a prompt with no context, the system falls back to a prompt that informs the model that no context was available and instructs it to answer from general knowledge, clearly stating that no specific context was available. This mode is intentionally not equivalent to a grounded RAG answer, because it reintroduces the possibility of unsupported model knowledge.
3. **Raw mode:** When context inclusion is disabled by the user, the query is passed directly to the LLM without any retrieved context or special instructions. Inclusion of this mode allows for direct comparison between grounded and non-grounded responses.

The grounded mode is the most important variant because it connects retrieval with the final generation step. The raw mode is useful for comparison, while the grounded mode and its fallback variant form the RAG answer-generation path.

4.4.2 Grounded Prompt Template

The grounded prompt combines the original user query with the retrieved context and guiding instructions for the LLM. It is assembled by the `build_grounded_answer_prompt()` function using the following structure:

You are a retrieval-grounded assistant.
Answer only from the retrieved context when it is provided.
Do not use outside knowledge.
Cite supporting claims inline using source ids like [S1] and [S2].
If the retrieved context is missing or insufficient, say that clearly.

Question:

<user query>

Retrieved context:

[S1] filename=<name> document_id=<id> chunk_index=<n>
<chunk content>

[S2] filename=<name> document_id=<id> chunk_index=<n>
<chunk content>

Answer:

Figure 4.3: The grounded-mode prompt template. The first few lines constrain the model to the attached context, enforce citations and require the model to acknowledge when it had insufficient context. Each retrieved chunk is rendered as a block with its source identifier and metadata.

For the chunks the `filename`, `document_id` and `chunk_index` metadata are placed before the content rather than in a separate JSON block, because the LLM can parse and understand this format more easily.

4.4.3 Source Attribution

Chunks in the prompt are prefixed with a source identification token in the [S1], [S2]... format rather than internal chunk IDs. This allows the LLM to cite sources inline in the generated answer, providing clean citations. The source identifiers correspond to the order of retrieved chunks. Internally the application assigns the chunks themselves source identifiers which match the implementation of the [S1] chunk token format in the prompt, so the final answer can be traced back to the original chunk and therefore the metadata from SQLite.

4.5 API Layer and Internal Application State

The FastAPI application is created through a `create_app()` function that initializes the application. During startup, a context manager takes care of initializing the SQLite schema (tables for documents, chunks, jobs, and the FTS5 virtual table), makes sure the storage directory exists, builds the Qdrant instance and verifies connectivity, and initializes the embedding and generation clients. The state of these components is stored on `app.state` and accessible across the application, including the API route handlers.

The three direct retrieval endpoints (`/api/query/dense`, `/api/query/lexical`, and `/api/query/hybrid`) share the `_run_retrieval_query()` helper, which builds the selected retriever, executes the search, and formats the response. The `/api/query/answer` endpoint uses the same retriever-building logic, then passes the retrieved sources to the generation service and returns the generated answer with the source list. The additional `/api/query/prompt-debug` endpoint builds the full prompt using the same logic as the

answer generation endpoint, but without calling the LLM, this is useful for verification of prompt construction and retrieval quality during development.

The document upload endpoint (`/api/documents/upload`) accepts and validates a file upload, saves it to disk, creates a document record in SQLite, and starts the background indexing job, returning the response immediately with the document and job IDs.

4.6 Frontend

The application includes a minimal web-based user interface implemented as a single static HTML page with embedded CSS and JavaScript. This frontend is served at the root endpoint (`/`) directly from the FastAPI application.

The interface provides two views: The **query view** with a query input area, retrieval mode selector (dense/lexical/hybrid), context inclusion toggle, and an area to display the generated answer along with the cited sources. and A **documents view** with a filefield for uploading documents and a list of uploaded documents with their status indicators (pending, running, success, fail).

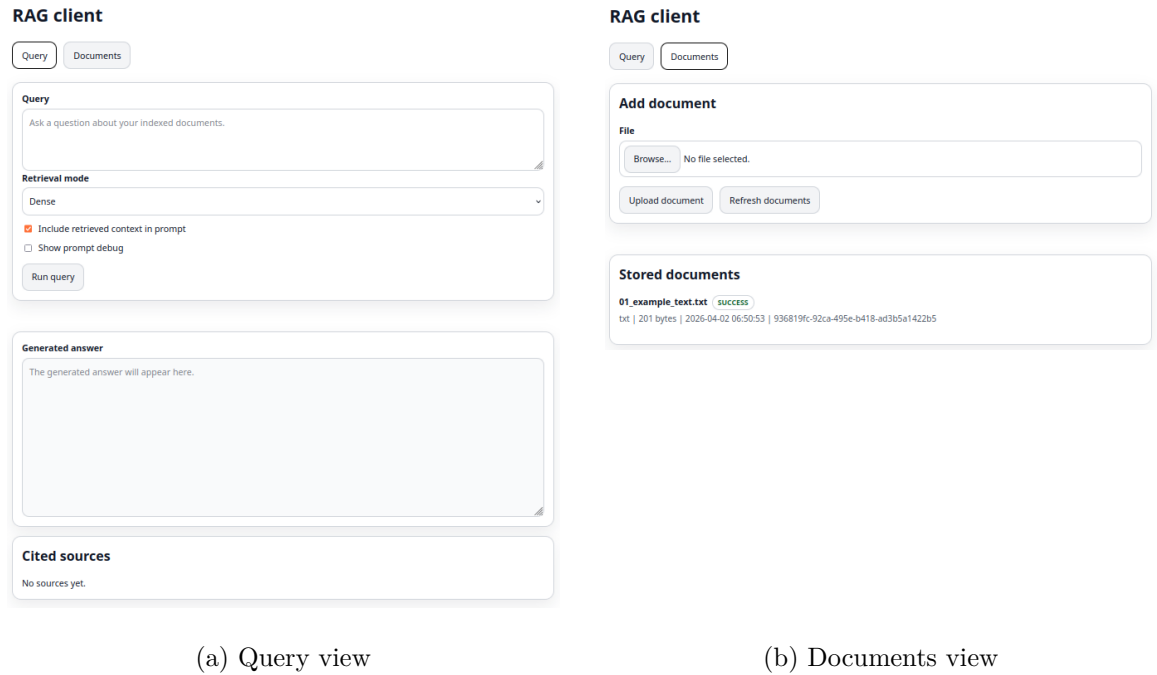


Figure 4.4: Web frontend of the implemented RAG application. Left: query view for submitting questions, selecting the retrieval mode, and inspecting generated answers with cited sources. Right: documents view for uploading source documents and checking their indexing status.

4.7 End-to-End Example

To further illustrate the implementation and demonstrate how the subsystems described in the previous sections work together, this section presents a walkthrough of a single user query from submission to answer generation. The knowledge base used in this example

consists of two documents each describing BitSwan automations and BitSwan workspaces. This is a domain-specific topic which is not widely covered in public web sources, so the language model is unlikely to be able to answer from its pre-trained knowledge. The answer should therefore be grounded in the retrieved context.

The example query was chosen so that it doesn't copy a section title or all of the relevant keywords from the documentation. Instead, it asks how promotion of automations in a BitSwan workspace works:

Example Query

"How does code move from development to testing and finally to the live deployment in a BitSwan workspace, and what should I know about the differences between these environments?"

Step 1 – Retrieval. The `HybridRetriever` runs the dense and lexical retrievers independently and combines their ranked outputs using Reciprocal Rank Fusion. Table 4.1 summarises the most important retrieved chunks for the example query.

Dense retrieval mainly favoured chunks that were semantically related to the deployment lifecycle, such as the platform overview and workspace deployment concepts. Lexical retrieval gave higher scores to chunks containing exact words from the query, especially words such as "deployment", "development", "staging", "production", and `automation.toml`. The hybrid retriever then combined the chunks choosing ones which scored best across both retrieval methods. As a result, the final context contained the main chunk describing the four deployment stages, together with multiple supporting chunks.

Rank	Dense retrieval	Lexical retrieval	Hybrid retrieval
1	Workspace and automation structure	Automation isolation and legacy deployment notes	Deployment stages: <code>live-dev</code> , <code>dev</code> , <code>staging</code> , <code>production</code>
2	Agent startup guide overview	Deployment stages and runtime stage detection	URL and deployment ID conventions
3	Automations Operations Center (AOC) and workspace management overview	Secret groups and deployment verification	Stage-specific secrets and <code>automation.toml</code> configuration
4	General automation architecture	Agent guide introduction	GitOps deployment and promotion workflow

Table 4.1: While the top chunks retrieved by each method differ, the hybrid result brought the most relevant stage description and added supporting chunks needed to explain secrets, URLs, and promotion between environments.

Step 2 – Prompt assembly. Fused chunks are passed to `build_grounded_answer_prompt()`, which inserts the user query and the retrieved source blocks into the template. Each chunk is labelled with a source identifier "[S1], [S2], [S3] ..."

Step 3 – Generation and response. The prompt is sent to the configured LLM through LiteLLM. Since the relevant chunks describe the four BitSwan stages and their differences, the model can produce a grounded answer such as:

Grounded Answer Example

Code moves through four deployment stages in a BitSwan workspace, each implemented as an independent deployment with its own containers, secrets, and service instances, including a separate CouchDB instance [S1].

The progression is:

- **live-dev**: active development with source code mounted to `/app/` and auto-reload enabled [S1]
- **dev**: a built-image development snapshot that shares secrets with **live-dev** via `automation.toml` [S1][S3]
- **staging**: pre-production testing with its own stage-specific secrets [S1]
- **production**: live deployment using a built image, with production secrets and no stage suffix in the public URL [S1]

At runtime, the platform injects the `BitSwan_AUTOMATION_STAGE` variable, which is used by the entrypoint to switch behavior between environments [S1]. The workspace name defines the namespace for all deployed services [S4].

Key differences between environments:

- **Isolation**: each stage runs as a fully independent deployment with separate infrastructure and data [S1]
- **Execution mode**: **live-dev** supports hot reload, while the other stages run built images [S1]
- **Secrets**: credentials are configured per stage; **dev** may share with **live-dev**, while **staging** and **production** use separate groups [S1][S3]
- **URLs**: only production omits the stage suffix in its public endpoint [S1]

The API response returns the generated answer together with the retrieved chunks and their source identification. This keeps the mapping between cited claims in the answer and the document chunks they are based on traceable and inspectable in the frontend.

Why hybrid retrieval helped. This query helps to demonstrate the benefit of combining lexical and dense retrieval. Dense retrieval was able to find workspace, deployment and architecture related results, even when the query didn't exactly match the keywords in the documents. However, some results were too general to answer the question on their own. Lexical retrieval, on the other hand, strongly matched chunks containing exact stage and deployment terminology, but also retrieved some partially relevant chunks that contained the individual keywords such as "deployment" or "automation". The hybrid retriever improved the final context by promoting chunks that were both semantically related and contained some lexical matches. This produced a context that was more fit for answering the question, which in turn allowed the LLM to generate a more complete answer.

4.8 Deviations from the Design

During implementation, some deviations from the original design were made based on practical experience:

1. **Lexical search matching**: The original design specified AND matching for all query terms which is very strict. In practice there is a low probability of a multi-word natural

language query matching a chunk with all terms. Therefore the implementation adds an OR fallback: If the strong-match AND query fails to return any results, the retriever automatically falls back to a weaker OR query that allows any of the terms to match individually. This returns considerably more results for queries at the cost of some precision, introducing more noise into the retrieved context, while still proving worthwhile in providing useful information for the LLM.

2. **No-context prompt mode:** Initial design specified that when retrieval returns no results, the system should instruct the model to refuse answering. This was changed so the LLM receives acknowledging the lack of context and is allowed to answer from general knowledge. This reflects a more realistic behavior these systems would have in production.
3. **Extraction error handling:** Original design returned extraction errors as synchronous HTTP 422 response during upload. The implementation moves all extraction errors into the asynchronous job pipeline: upload succeeds immediately, and extraction failures are reported through the job status endpoint.
4. **Background task mechanism:** FastAPI's built-in `BackgroundTasks` FastAPI's built-in `BackgroundTasks` was chosen for its simplicity and sufficiency for a small-scale application, lacking retry mechanism and persistence across system restarts, it still provides the benefits of asynchronous processing without too much overhead.

4.9 Known Limitations and Future Work

- **No document lifecycle management:** Documents can be uploaded and indexed but not deleted or re-indexed without re-uploading.
- **Single-worker limitation:** The background task model does not support parallel indexing or retry handling.
- **No streaming:** Answers are returned as complete responses. Streaming token-by-token via Server-Sent Events would improve perceived latency for longer answers.
- **No authentication:** The application is designed for local single-user use. Multi-user deployment would require authentication, authorization and isolation.

Chapter 5

Testing

This chapter evaluates the implementation from two complementary views. Firstly, automated tests verify that the main implementation components work as expected at the level of individual modules and API endpoints. Secondly, controlled experimentation tests whether the retrieval and chunking choices fulfill expectations on small datasets. The goal is not to compete with benchmark-level performance, but rather to showcase the functionality and prove the effectiveness of the RAG system on a small scale.

5.1 Testing Goals and Environment

The tested system is the FastAPI RAG application implemented in Chapter 4, which consists of the document ingestion pipeline, retrieval system, and answer generation. The tests therefore focus on:

- validation of individual components such as extraction, chunking, configuration validation, retrieval fusion, and prompt construction,
- correct persistence in the indexing pipeline,
- API behavior for upload, querying, prompt debugging, job inspection, and health checks,
- retrieval quality and answer grounding on examples.

Automated tests are designed to avoid external services when possible, making them faster and not requiring a running LLM API. On the other hand, the retrieval experimentation tests the system as if it was a black box, using the API endpoints and a real LLM to verify the system performance.

5.2 Automated Tests

The implementation contains a collection of automated tests. The suite of these tests is based on Python's `unittest` and can be run with the `make test` command, which runs:

```
uv run python -m unittest discover -s tests -p "test_*.py"
```

This command discovers and runs all test modules in the `tests` directory that match the pattern `test_*.py`. The tested areas are summarized in Table 5.1.

Area	Verified behavior
Configuration	Required settings, invalid values, vector size, chunk-size constraints, and provider-key validation.
Extraction and chunking	TXT, Markdown, and PDF extraction, invalid inputs, determinism of recursive splitting, and overlap behavior.
Storage and indexing	SQLite template creation, FTS5 table creation, document metadata, chunk persistence, job records, partial embedding failures, and Qdrant indexing failures.
Retrieval	Dense retrieval, SQLite FTS5 lexical retrieval, strict AND queries, OR fallback, tie-breaking, and RRF-based hybrid fusion.
Generation	Grounded prompt construction, no-context prompts, context-disabled prompts, LiteLLM response mapping, and source identifiers.
API layer	Upload and listing endpoints, job and health endpoints, dense/lexical/hybrid query endpoints, answer generation, prompt debugging, and serving of the local UI.

Table 5.1: Main areas covered by the automated test suite.

The API-level tests use FastAPI’s `TestClient`. They verify requests and responses without the need to open an external HTTP socket. This makes it possible to test the API while still using temporary local database and mocked adapters. For example, the tests check that retrieval responses include chunk content and relevant metadata, or that the answer responses return both generated text and source citations correctly.

5.3 Controlled Retrieval Evaluation

While the automated tests verify the implementation with the combined use of unit and integration tests, they do not show if the dense, lexical, or hybrid retrieval behave differently in different query situations. For this reason the retrieval evaluation assesses both ingestion and generation pipelines by running real queries against a small, controlled knowledge base and inspecting the results. The documents in this base are intentionally simple: some contain exact keywords, some describe the same concept without the relevant keyword, and some mix both semantic and lexical content. This makes evaluation process especially simple, the documents are easily manageable and results are easily comparable without a need for more complex benchmarks.

The retrieval modes are compared with a `top_k` number of results, using the same knowledge base for retrieval. Every query is built around a specific relevant document group, defined and ingested before the run. Table 5.2 summarizes the ingested document groups and why they were included in the evaluation.

Document group	Representative gested files	in-	Purpose and contents
Cat – keyword	Cat lexical file		Contains explicit occurrences of the word <code>cat</code> . It checks whether lexical retrieval ranks a direct keyword match correctly.
Cat – semantic	Cat semantic file		Describes cat-related behaviour without relying on the exact keyword. It tests whether dense retrieval can find a relevant document from meaning rather than term match.
Animal contrast	Dog semantic and bird lexical files		Provides related but distinct animal examples. These files check whether retrieval separates nearby semantic topics and still handles exact bird keywords.
Vector database	Vector-database and keyword-only database files		Separates a relevant technical file about vector databases from a noisy general database vocabulary file.
BitSwan documentation	Agent guide, development guide, and pipeline walk-through		Larger documentation-style inputs with domain-specific knowledge covering development workflow, automation configuration, and development(dev/live-dev), staging, production deployment behaviours. They are used mainly for chunking and grounded-answer checks.

Table 5.2: Document groups used in the controlled retrieval and chunking evaluation.

The recorded metrics are:

- **Hit@1**: relevance of the top retrieval result
- **Recall@3**: whether at least one relevant result appears among the first three results,
- **Qualitative note**: a short exploration and assessment of the results of the ranking, especially where lexical and dense retrieval disagree.

The following table explains the intents and expectations of each query category tested:

Case type	Example query intent	Expected observation
Exact lexical match (3x)	Query contains terms such as cat , bird , or vector database .	Lexical retrieval should rank files containing the exact terms highly.
Semantic para-phrase	Query describes a pet with whiskers and purring without using the word cat .	Dense retrieval should be more useful than exact keyword matching.
Hybrid signal	Query combines semantic intent with terms such as feline or cat .	Hybrid retrieval should promote chunks supported by both semantic and lexical evidence.
Noisy keyword	Query contains broad terms such as database where multiple chunks may match.	Lexical retrieval may return keyword-related noise; hybrid ranking should reduce this when semantic context picks only the most relevant matches.

Table 5.3: Controlled retrieval cases used to compare dense, lexical, and hybrid modes.

This experiment is intentionally small, however it clearly highlights the advantages of using hybrid retrieval. Dense search helps when the query is phrased differently from the source document. Lexical search is expected to help with finding exact terminology. And while the hybrid retrieval is a complementary combination of both searches, being the most robust, the experiment encounters results where hybrid fusion did not compensate for a weak set of candidate chunks.

The experiment was executed through the running API with the use of the app UI and with OpenAI embedding provider enabled. Where answer generation was evaluated, the configured `gpt-5.4-mini` model was used through LiteLLM. The `top_k` value was set to 3 and the results were then stored for evaluation. In the small source document collection all three modes found the expected document at rank 1 for each of the six aforementioned queries, giving the aggregate result in Table 5.4. The results in this table do not prove the modes equivalent, and with the scale of the experiment it is not feasible to prove or disprove such hypotheses. Rather it verifiably showcases that the implemented endpoints return relevant information with this small scale of source documents.

Mode	Hit@1	Recall@3	Observation
Dense	6/6	6/6	Found the expected document for semantic animal descriptions and technical Qdrant wording.
Lexical	6/6	6/6	Strong on exact-keyword cases.
Hybrid	6/6	6/6	Preserved the relevant first result while returning candidate chunks from both dense and lexical evidence.

Table 5.4: Aggregate result of the controlled retrieval evaluation.

Table 5.5 shows three illustrative query results. The semantic cat-themed query retrieved the paraphrased cat document first, but the top three also contained a dog document, showing that even the successful retrieval may introduce related but not fully relevant context. In this case the dog document was introduced due to lack of other relevant chunks in the knowledge base. It instead serves as a good evidence of a correct embedding and

retrieval. The database dense query retrieved the vector-database document first, while the keyword-heavy database query retrieved the intentionally noisy database file first.

Query	Retrieved chunk excerpts
<i>Which animal has whiskers, purrs, naps in sunny places, and stalks laser dots?</i>	Dense top 3: “Quiet apartment pet profile – This small four-legged companion has soft fur, sharp retractable claws, long whiskers, and a habit of purring when relaxed.” “Feline behavior notes – A cat may appear distant, but this graceful feline often forms routines around the people it trusts.” “Loyal family companion – This energetic household animal greets people at the door, wags its tail, learns commands, enjoys long walks, and may bark when excited or protective.”
<i>Which document explains Qdrant embeddings HNSW dense retrieval and vector database search?</i>	Dense top result: “Vector database overview – Qdrant is a vector database used for dense retrieval and semantic search. It stores embeddings and metadata, and it commonly uses HNSW indexing.”
<i>database export client admin panel payroll invoice</i>	Lexical top result: “Database note – This file is mostly for lexical matching. Keywords: invoice, payroll, contract, database, export, client, admin panel.”

Table 5.5: Verbatim excerpts from selected raw retrieval responses.

5.4 Chunk Size Experiment

Chunking affects both indexing and retrieval. Very small chunks may produce precise matches but lose context needed for answer generation. Although very large chunks can introduce irrelevant context and reduce retrieval precision, they may still hold advantages compared to no-retrieval approach that attaches complete source documents to prompt. Ever-increasing context window sizes make such brute-force prompting technically possible, they do not ensure that the model will reliably utilize the relevant parts of the document. This approach also becomes inefficient, especially in the cases where the documentation for the most complex systems is still too extensive. Prompt size, cost and latency are also naturally affected by this. Correct use of a RAG has potential to speed-up and cheapen the whole LLM generation process. To explore the benefits and tradeoffs of varying chunk sizes, the system was evaluated with three chunk-size configurations:

Chunk size	Overlap	Purpose of the configuration
500	50	Smaller chunks, improved precision, lost context tradeoff
1000	150	Default configuration used by the system.
1500	200	Larger chunks, expected to preserve more context but at risk of less focused matches.

Table 5.6: Chunking configurations for the sensitivity experiment.

The experiment used larger files, taken from the documentation describing functionality, structure and behavior of BitSwan - an open-source tool used to build data pipelines and automations. The same documents as in the end-to-end example 4.7. For each con-

figuration, the same collection of the source documents is re-indexed and the same query set is executed. Number of produced chunks, average chunk length and retrieval quality are recorded and compared. The results are compared and the default parameter size is evaluated against alternatives.

The experiment was run on three BitSwan documentation files: `AgentGuide(2).md`, `DEVELOPMENT_GUIDE.md`, and `PIPELINE_WALKTHROUGH.md`. Each chunking configuration used an isolated SQLite database and qdrant collection, while the same three hybrid queries were executed after indexing.

Chunk size	Overlap	Chunks	Avg. length	Observation
500	50	177	376.0	Most precise segmentation, but the retrieved snippets often began or ended inside tables or sentences.
1000	150	86	819.9	Balanced result; retrieved chunks contained enough local context with some longer sections still being split.
1500	200	50	1350.7	Fewer chunks and fastest indexing in this run, but each retrieved chunk mixed more surrounding topics.

Table 5.7: Chunking experiment results on the BitSwan documentation.

The default 1000/150 configuration remained a reasonable compromise for the used documentation. While still cutting off some content, better results would be achieved by increasing the `top_k` number of retrieved chunks rather than by chunk size scaling. The 500/50 setting increased the number of chunks by more than two times compared with the default configuration, which gives retrieval more potential chunks, but often splitting markdown tables, or longer paragraphs. The 1500/200 setting reduced the number of chunks to 50, but the returned chunks contained enough text that the final prompt often received more surrounding material than was needed for the question.

5.5 Grounded Answer and Source Traceability Check

The final user-visible behavior of the application is not only retrieval, but retrieval-grounded answer generation. This example checks whether retrieved chunks are correctly presented in-prompt to the LLM and whether the answer can be traced back to returned sources.

A small set of domain-specific questions was used to evaluate the system with queries that cannot be reliably answered by standalone LLM. The selected queries focus on information that is not widely available in public training data and is unlikely to be stored in the model’s parameters. Each question is evaluated in two prompt modes:

- **Context enabled:** `include_context_in_prompt=true`, where retrieved chunks are inserted into the prompt with source labels.
- **Context disabled:** `include_context_in_prompt=false`, where the model receives only the raw user query.

The `/api/query/answer` endpoint is used to generate the final answer. The generated answer is then recorded, and the context-enabled and no-context answers are compared. The check also verifies the correctness of source identifiers, the relevance of retrieved chunks to the query, and whether disabling retrieved context makes the answer less specific or traceable.

The source-traceability check was run with three questions about BitSwan using hybrid retrieval with `top_k=4`. All three context-enabled answers cited source identifiers that were present in the returned source list. The most illustrative example was the deployment-stage question. Because the answer uses labels such as `[S1]`, the returned source list is shown before the answer is decomposed.

Source	ID	Document	Chunk	Relevant retrieved content
1	[S1]	AgentGuide(2).md	2	Stage table describing live-dev , dev , staging , and production , including deployment independence and runtime stage detection.
2	[S2]	AgentGuide(2).md	1	Project structure showing automation.toml , app/ , image files, and the entrypoint script used for stage-specific logic.
3	[S3]	AgentGuide(2).md	15	CouchDB stage isolation: separate databases per stage, no automatic data flow, and stage-specific credentials.
4	[S4]	DEVELOPMENT_GUIDE.md	16	Example automation.toml secret-group configuration for dev , staging , and production .

Table 5.8: Numbered source list returned for the deployment-stage answer.

Question: How does code move from live development to staging and production in BitSwan, and what changes between environments?	
Context-enabled answer	No-context answer
Code moves through BitSwan’s stages as separate deployment stages: live-dev , dev , staging , and production . Each stage is an independent deployment with separate containers, secrets, service instances, and CouchDB where enabled [S1]. live-dev mounts source code to /app/ , while dev , staging , and production use built images [S1] [S2]. The answer also identifies stage-specific behavior, runtime detection through BITSWAN_AUTOMATION_STAGE , and CouchDB isolation [S1] [S3].	In BitSwan, code typically moves through a controlled promotion path from live development to staging and then to production. Live development is used for frequent changes and quick testing, staging is described as a production-like environment for release validation, and production serves real users. The answer says that code should usually stay the same, while configuration changes between environments, such as database endpoints, API keys, feature flags, logging levels, external service URLs, resource limits, and security settings.

Table 5.9: Side-by-side showcase of the same question with and without retrieved context.

No.	Answer statement	Citations	Retrieved support
1	BitSwan uses four independent deployment stages: <code>live-dev</code> , <code>dev</code> , <code>staging</code> , and <code>production</code> . Each stage has separate containers, secrets, and service instances.	[S1]	Stage table
2	In <code>live-dev</code> , source code is mounted to <code>/app/</code> ; in <code>dev</code> , <code>staging</code> , and <code>production</code> , built images are used.	[S1] [S2]	Stage table and project structure
3	<code>live-dev</code> is intended for active development with hot reload, while <code>dev</code> is a development snapshot build.	[S1]	Stage table
4	<code>staging</code> is used for pre-production testing and <code>production</code> serves live traffic with production secrets and no <code>-{stage}</code> URL suffix.	[S1]	Stage table
5	The runtime stage is exposed through <code>BITSWAN_AUTOMATION_STAGE</code> , which allows endpoint scripts to switch behavior.	[S1]	Runtime stage text
6	When CouchDB is enabled, every stage receives a dedicated CouchDB instance, so data and credentials do not automatically carry across stages.	[S3]	CouchDB isolation text

Table 5.10: Traceability of the context-enabled answer for the deployment-stage question.

The comparison shows that the context-enabled answer is more specific and traceable. It names the exact BitSwan stages, distinguishes source-mounted live development from built-image stages, and cites the retrieved chunks that support these claims. The no-context answer remains plausible, but it is generic: it describes a common deployment workflow that the BitSwan workflow resembles, since BitSwan aims to be familiar and intuitive for the user. The claims generally range from plausible and correct assumptions to clearly incorrect statements.

5.6 Limitations of the Evaluation

The evaluation has multiple important limitations. The automated tests verify functionality, integration, and behavior, but do not prove the correctness or quality of retrieval and ingestion. The retrieval experiment uses a small controlled knowledge base, not a standardized benchmark such as BEIR. The query set is manually constructed and therefore useful for debugging and explanation, but it is not large enough for statistical claims.

The chunking experiment only tests technical documents; different categories of documents, such as legal contracts, medical documents, source-code documentation or short FAQ entries may reach peak performance with different chunk sizes and overlap values. Finally, the answer-generation check depends on the configured LLM and is evaluated qualitatively. Because it lacks independent evaluation, the results may be affected by author bias. It should be interpreted as evidence that the implementation provides traceability by supplying sufficient context and guidance for the LLM.

Chapter 6

Conclusion

This thesis explored the design and implementation of a Retrieval-Augmented Generation application for querying a managed knowledge base. The resulting prototype supports document ingestion for PDF, plain text, and markdown files, recursive chunking, SQLite metadata storage, SQLite FTS5 lexical search, Qdrant dense-vector storage, dense, lexical, and hybrid retrieval, prompt construction answer generation through LiteLLM, a REST API, and a simple local web interface.

The implementation fulfills the main requirements defined in the design chapter, It demonstrates that a custom application can implement them without any use of a large RAG framework. This custom approach required more implementation work, but enabled the individual system components inspectable and testable.

The testing chapter verified the prototype from two complementary perspectives. Automated tests covered configuration, extraction, chunking, storage, indexing, retrieval, prompt construction, answer generation and the fastAPI API. Experiments then evaluated the system through this API. These experiments compared retrieval modes on a small set of labeled source documents, evaluated the effect of different chunking parameters, and checked the quality of generated answers and traceability of statements to returned source chunks. The results showed that the implemented system can retrieve relevant context, cite retrieved chunks, and produce more reliable answers.

The work also has clear limitations. The prototype is built for a single-user use, does not hold context of the conversation across multiple prompts, does not support document deletion, re-indexing, worker queue, retry handling and answer streaming. The evaluation resorted to use of simple controlled examples rather than a standardized benchmark, therefore not measuring the retrieval performance itself. It only demonstrates that the implemented architecture works end-to-end.

Future work should focus on document management, larger retrieval evaluation, enabling conversation over multiple questions, multi-user deployment and answer streaming. Another possible continuation is to pivot main parts of the implementation to serve as a tool for an LLM-based agent or coding assistant. This direction is motivated by the recent growth of agent-based LLM workflows, which became especially relevant during the development of this application. Although many current models and assistants can already retrieve external information through built-in tools, these tools may be less suitable when the required context is local, private or domain-specific. In such a setting, the ingestion and retrieval pipelines could provide task-specific context on demand, including project documentation, implementation notes, or other information that may not be available to the model in its parameters. Such a tool could lower the amount of data in the context

window, or help find the context which a regular search tool might miss, especially with the semantic search already integrated. This could support agents performing implementation, maintenance, or debugging tasks.

Bibliography

- [1] BOJANOWSKI, P.; GRAVE, E.; JOULIN, A. and MIKOLOV, T. Enriching Word Vectors with Subword Information. *Transactions of the Association for Computational Linguistics* online. MIT Press, 2017. Available at: https://doi.org/10.1162/tac1_a_00051. [cit. 2026-04-08].
- [2] BROWN, T. B.; MANN, B.; RYDER, N.; SUBBIAH, M.; KAPLAN, J. et al. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems* online. Curran Associates, Inc., 2020, vol. 33, p. 1877–1901. Available at: <https://proceedings.neurips.cc/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf>. [cit. 2025-12-31].
- [3] CER, D.; YANG, Y.; KONG, S. yi; HUA, N.; LIMTIACO, N. et al. Universal Sentence Encoder for English. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations* online. Brussels, Belgium: Association for Computational Linguistics, 2018, p. 169–174. Available at: <https://doi.org/10.18653/v1/D18-2029>. [cit. 2026-04-07].
- [4] CORMACK, G. V.; CLARKE, C. L. A. and BÜTTCHER, S. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In: *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval* online. ACM, 2009, p. 758–759. Available at: <https://doi.org/10.1145/1571941.1572114>. [cit. 2026-04-08].
- [5] JÉGOU, H.; DOUZE, M. and SCHMID, C. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* online. IEEE, 2010, vol. 33, no. 1, p. 117–128. Available at: <https://doi.org/10.1109/TPAMI.2010.57>. [cit. 2026-04-07].
- [6] JOHNSON, J.; DOUZE, M. and JÉGOU, H. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* online. IEEE, 2021, vol. 7, no. 3, p. 535–547. Available at: <https://doi.org/10.1109/TBDATA.2019.2921572>. [cit. 2026-04-07].
- [7] LEWIS, P.; PEREZ, E.; PIKTUS, A.; PETRONI, F.; KARPUKHIN, V. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: *Advances in Neural Information Processing Systems* online. Curran Associates, Inc., 2020, vol. 33, p. 9459–9474. Available at: <https://proceedings.neurips.cc/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf>. [cit. 2026-01-03].
- [8] MALKOV, Y. A. and YASHUNIN, D. A. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* online. IEEE, 2018,

- vol. 42, no. 4, p. 824–836. Available at: <https://doi.org/10.1109/TPAMI.2018.2889473>. [cit. 2026-04-07].
- [9] MALLEN, A.; ASAI, A.; ZHONG, V.; DAS, R.; KHASHABI, D. et al. When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* online. Toronto, Canada: Association for Computational Linguistics, 2023, p. 9802–9822. Available at: <https://doi.org/10.18653/v1/2023.acl-long.546>. [cit. 2026-01-03].
 - [10] MIKOLOV, T.; CHEN, K.; CORRADO, G. and DEAN, J. Efficient Estimation of Word Representations in Vector Space. In: *Proceedings of the International Conference on Learning Representations (ICLR) Workshops*. 2013. Available at: <https://openreview.net/forum?id=idpCd0WtqXd60>. Workshop paper.
 - [11] PENNINGTON, J.; SOCHER, R. and MANNING, C. GloVe: Global Vectors for Word Representation. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* online. Doha, Qatar: Association for Computational Linguistics, 2014, p. 1532–1543. Available at: <https://doi.org/10.3115/v1/D14-1162>. [cit. 2026-04-08].
 - [12] RAFFEL, C.; SHAZEER, N.; ROBERTS, A.; LEE, K.; NARANG, S. et al. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research* online, 2020, vol. 21, no. 140, p. 1–67. Available at: <https://jmlr.org/papers/v21/20-074.html>. [cit. 2026-01-03].
 - [13] REIMERS, N. and GUREVYCH, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* online. Hong Kong, China: Association for Computational Linguistics, 2019, p. 3982–3992. Available at: <https://doi.org/10.18653/v1/D19-1410>. [cit. 2026-01-03].
 - [14] ROBERTS, J. How Powerful are Decoder-Only Transformer Neural Models? In: *2024 International Joint Conference on Neural Networks (IJCNN)* online. IEEE, 2024, p. 1–8. Available at: <https://doi.org/10.1109/IJCNN60899.2024.10651286>. [cit. 2026-04-27].
 - [15] SHUSTER, K.; POFF, S.; CHEN, M.; KIELA, D. and WESTON, J. Retrieval Augmentation Reduces Hallucination in Conversation. In: *Findings of the Association for Computational Linguistics: EMNLP 2021* online. Punta Cana, Dominican Republic: Association for Computational Linguistics, 2021, p. 3784–3803. Available at: <https://doi.org/10.18653/v1/2021.findings-emnlp.320>. [cit. 2026-01-03].
 - [16] THAKUR, N.; REIMERS, N.; RÜCKLÉ, A.; SRIVASTAVA, A. and GUREVYCH, I. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In: *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*. 2021. Available at: <https://openreview.net/forum?id=wCu6T5xFjeJ>.
 - [17] TOUVRON, H.; LAVRIL, T.; IZACARD, G.; MARTINET, X.; LACHAUX, M.-A. et al. LLaMA: Open and Efficient Foundation Language Models. 2023. ArXiv preprint.

- [18] VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L. et al. Attention Is All You Need. In: *Advances in Neural Information Processing Systems* online. Curran Associates, Inc., 2017, vol. 30. Available at: <https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf>. [cit. 2025-12-31].
- [19] WEI, J.; WANG, X.; SCHUURMANS, D.; BOSMA, M.; XIA, F. et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In: *Advances in Neural Information Processing Systems* online. Curran Associates, Inc., 2022, vol. 35, p. 24824–24837. Available at: https://proceedings.neurips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html. [cit. 2026-01-02].